

Linear models

Alessio Micheli

micheli@di.unipi.it



Dipartimento di Informatica
Università di Pisa - Italy

**Computational Intelligence &
Machine Learning Group**

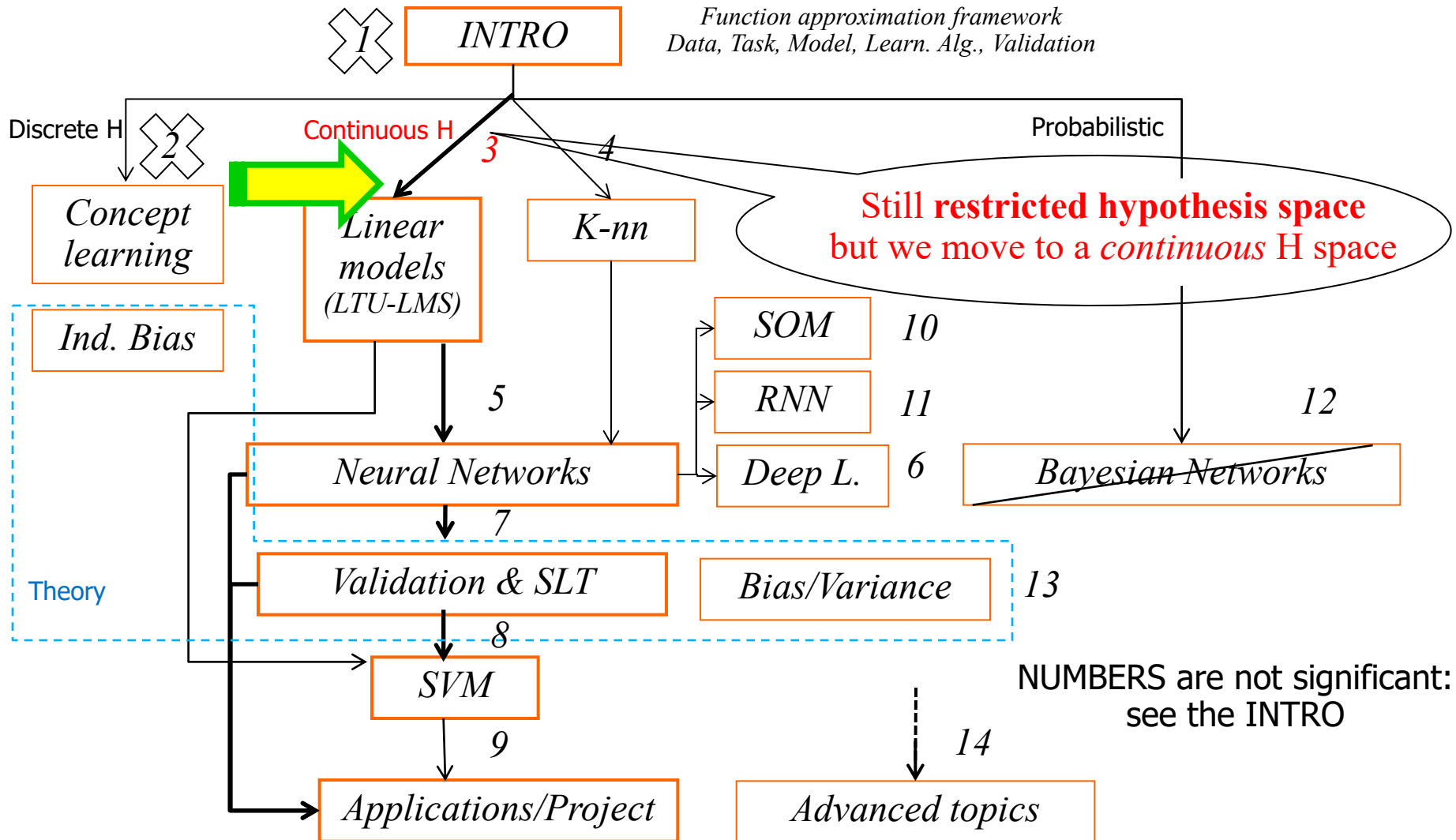
Sept, 2026

ML Course structure

Where we go

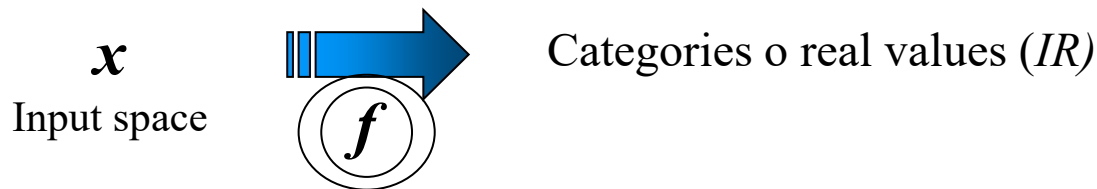


Dip. Informatica
University of Pisa



Tasks: Supervised Learning

- **Given:** Training examples as $\langle input, output \rangle = (x, d)$ (*labeled examples*)
for an unknown function f (known only at the given points of example)
 - Target value: desiderate value d or t or y ... is given by the teacher according to $f(x)$.
- **Find:** A *good* approximation to f (a hypothesis h that can be used for prediction on unseen data x' , i.e. that is able to generalize)



- Target d (or t or y): a categorical or numerical *label*
 - **Classification:** discrete value outputs:
 $f(x) \in \{1, 2, \dots, K\}$ *classes (discrete-valued function)*
 - **Regression:** real continuous output values (approximate a real-valued target function)

Both as a *task of function approximation*

A premise on **DATA** notation

Pattern	x_1	x_2	x_i	x_n
Pat 1	$x_{1,1}$	$x_{1,2}$		$x_{1,n}$
...				
Pat p	$x_{p,1}$	$x_{p,2}$	$x_{p,i}$	$x_{p,n}$
...				

X is a matrix $l \times n$

l rows, n columns

$p=1..l, \quad i=1..n$

We often need to omit some indices when the context is clear, e.g.:

- Each row, generic x (vector - bold), a row in the table: (input) example, pattern, instance, sample ,..., input vector, ...
- x_i or x_j (scalar): component i or j (given a pattern x , i.e. omitting p)
- x_p or x_i (vector – bold) p -th or i -th row in the table = pattern p or i
- $x_{p,i}$ (scalar) also as $(x_p)_i$: component i of the pattern p
or also, often, $x_{p,j}$ for the component j , etc.
- For the target y we will typically use just y_p with $p=1..l$ (the same for d or t)

Linear models

- **Regression**
- **Classification**

Linear models

The linear model has been the mainstay of statistics

- *"Despite the great inroads made by modern nonparametric regression techniques, linear models remain important, and so we need to understand them well". (Hastie)*

Plenty of studies and in many books (mathematics, statistics, numerical analysis, applicative fields, ML, ...)

- We start with the simplest form, linear in the input variables
- A baseline for learning (first: is it a linear problem?)

Regression

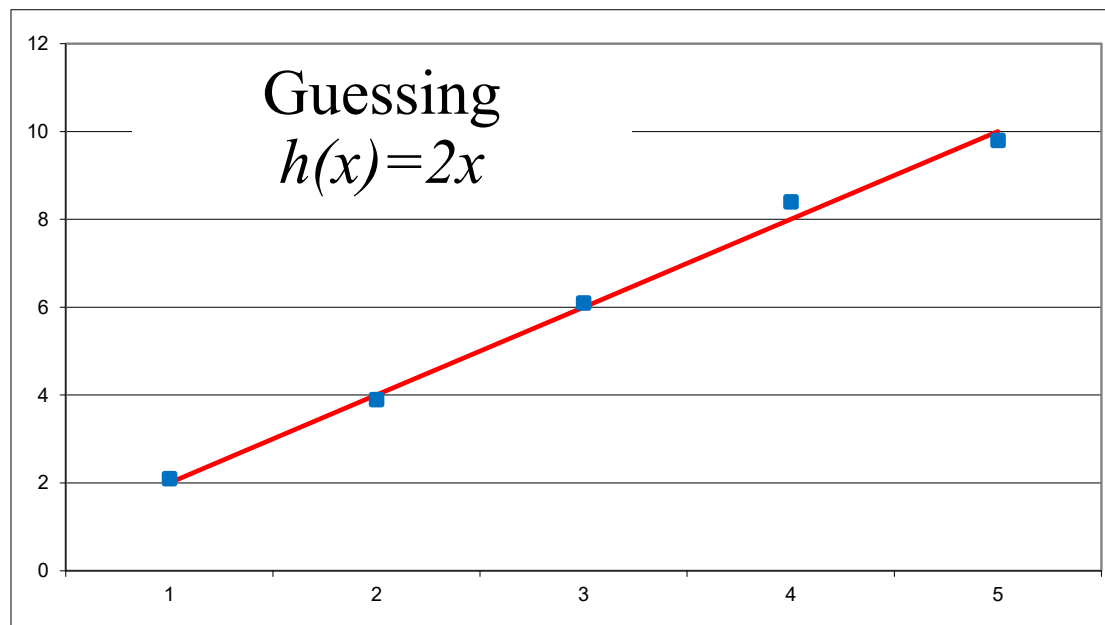
- **Just to see how formulate the *learning problem* as a **LMS** on the R_{emp}**
 - Then (next lecture) we will consider also the control of complexity
- **We formulate a first derivation in a simplified setting (univariate case)**

Repetita: regression: example

- Process of estimating of a real-value function on the basis of a finite set of noisy samples
 - known pairs $(x, f(x) + \text{random noise})$

Task (exercise): find f for the data in the following table:

x	target
1	2.1
2	3.9
3	6.1
4	8.4
5	9.8
...	...

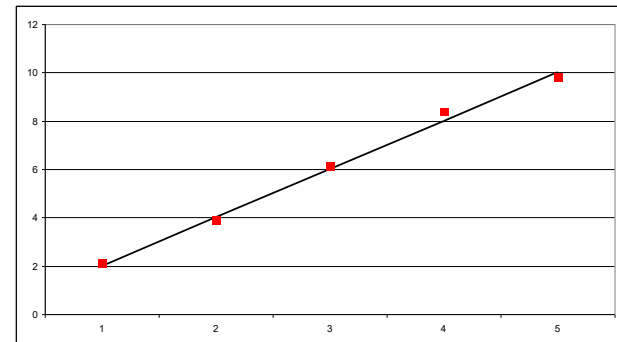


Now, we want to solve it (how to find w) in a «systematic» way

Univariate Linear Regression

- Univariate case, simple linear regression :
- We start with 1 input variable x , 1 output variable y $\xrightarrow{x} \boxed{h} \rightarrow$
- We assume a model $h_{\mathbf{w}}(x)$ expressed as $output = h_{\mathbf{w}}(x) = w_1 x + w_0$
- *where $\mathbf{w}$ are real-valued coefficients/free parameters (weights)*

- ***Fitting*** the data by a “straight line”



Univariate Linear Regression Learning!



Dip. Informatica
University of Pisa

$$h_w(x) = w_1 x + w_0$$

Since w are continuous values, this is an *infinite* hypothesis space but we have nice solution from classical math (going back to Gauss/Legendre ~1795!)

- Surprisingly we can “learn” by this basic tool
- Although simple it includes many relevant concept of modern ML and it is a basis of evolved methods in the field

Build it: Learning via LMS (I)

Training → find $\mathbf{w}$ such that minimize **error/empirical loss**
(best data fitting – on the training set with l examples):
i.e. we are now focusing on the R_{emp}

- **Given** a set of l training examples $(\mathbf{x}_p, y_p)$ $p=1..l$
- **Find:** $h_{\mathbf{w}}(x)$ in the form $w_l x + w_0$ (hence the values of $\mathbf{w}$) that *minimizes* the expected loss on the training data.

Build it: Learning via LMS (II)

For the loss we use the square of errors:

- Least (Mean) Square: Find $\mathbf{w}$ to *minimize* the residual sum of squares $[argmin_{\mathbf{w}} Error(\mathbf{w}) \text{ in } L_2]$:

$$Loss(h_{\mathbf{w}}) = E(\mathbf{w}) = \sum_{p=1}^l \overbrace{(y_p - h_{\mathbf{w}}(x_p))^2}^{\text{error}} = \sum_{p=1}^l (y_p - (w_1 x_p + w_0))^2$$

p runs over patterns/examples

where x_p is p -th input/pattern/example, y_p the target for p , w free par., l num. of examples

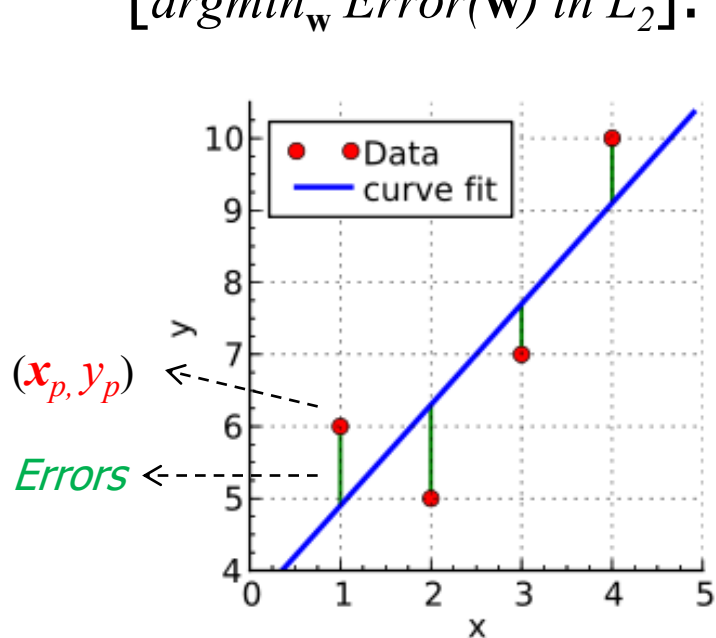
Note: to have the mean divide by l

On the notation: Indeed for the univariate case, with 1 variable: $x_p = x_{p,l} = (\mathbf{x}_p)_l$

Build it: Learning via LMS (III)

Why LMS to fit the data with h ?

- Least (Mean) Square: Find w to minimize the **residual sum of squares**
[$\operatorname{argmin}_w \operatorname{Error}(w)$ in L_2]:



$$h_w(x)$$

$$y = w_1 x + w_0 + \text{noise}$$

Different **blue** lines will have different **green** bars.

Minimizing the **green** bars (residuals / errors)

is a way to find the best approximation/fitting of the data (i.e. our $h_w(x)$ or **blue** line).

The squares of errors $E(w)$ quantify such **green** bars:

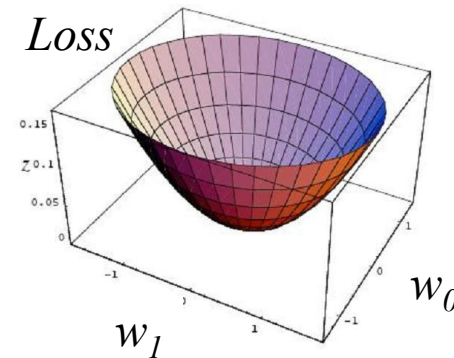
$$E(w) = \sum_{p=1}^l (y_p - h_w(x_p))^2$$

- The method of **least squares** is a standard approach to the approximate solution of over-determined systems, i.e., sets of equations in which there are more equations than unknowns.

How to solve? (I)

- Remember: local minimum as stationary point: the *gradient* is zero

$$\frac{\partial E(\mathbf{w})}{\partial w_i} = 0, \quad i = 1, \dots, \dim_input + 1$$



- For the simple Lin. Regr. (2 free parameters)

Search the $\mathbf{w}$
such that

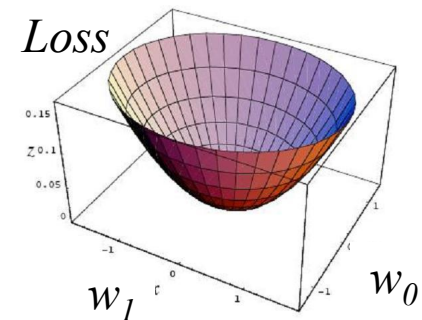
$$\frac{\partial E(\mathbf{w})}{\partial w_0} = 0 \quad \frac{\partial E(\mathbf{w})}{\partial w_1} = 0$$

How to solve? (II)

- For the simple Lin. Regr. (2 free parameters)

Search the $\mathbf{w}$
such that

$$\frac{\partial E(\mathbf{w})}{\partial w_0} = 0 \quad \frac{\partial E(\mathbf{w})}{\partial w_1} = 0$$



- Convex loss function $\rightarrow$ we have the following “direct” solution (no local minima)
(*just to know that it exist! Do not memorize*)

$$w_1 = \frac{\sum_{p: 1 \rightarrow l} x_p y_p - \frac{1}{l} \sum x_p \sum y_p}{\sum x_p^2 - \frac{1}{l} (\sum x_p)^2} = \frac{\text{Cov}[x, y]}{\text{Var}[x]}, \quad \begin{matrix} w_0 = \bar{y} - w_1 \bar{x} \\ \downarrow \qquad \qquad \downarrow \\ \frac{1}{l} \sum_{p \rightarrow l} y_p \quad \quad \frac{1}{l} \sum_{p \rightarrow l} x_p \end{matrix}$$

Exercise: compute w_0 and w_1 according to the next slide results (extended to l patterns)

Where l is num. of examples

Compute the gradient for 1 (each) pattern p



Dip. Informatica
University of Pisa

Redo this by yourself as an *Exercise*

Basic rules:

$$\frac{\partial}{\partial w} k = 0, \frac{\partial}{\partial w} w = 1, \frac{\partial}{\partial w} w^2 = 2w$$

$$\frac{\partial (f(w))^2}{\partial w} = 2f(w) \frac{\partial (f(w))}{\partial w}$$

Der. sum = sum of der.

We will call $(y - h(x))$
"delta"

$$\frac{\partial E(\mathbf{w})}{\partial w_i} = \frac{\partial (y - h_{\mathbf{w}}(x))^2}{\partial w_i} =$$

$$= 2(y - h_{\mathbf{w}}(x)) \frac{\partial (y - h_{\mathbf{w}}(x))}{\partial w_i} = 2(y - h_{\mathbf{w}}(x)) \frac{\partial (y - (w_1 x + w_0))}{\partial w_i}$$

$$\frac{\partial E(\mathbf{w})}{\partial w_0} = -2(y - h_{\mathbf{w}}(x))$$

$$\frac{\partial E(\mathbf{w})}{\partial w_1} = -2(y - h_{\mathbf{w}}(x)) \cdot x$$

Then we will sum up for l patterns $(x_p, y_p) \dots$

And we will extend to multidimensional x and $\mathbf{w}$ ($n=1$ here) ...

Linear model: notation for multidimensional inputs

- Assuming column vector for $\mathbf{x}$ and $\mathbf{w}$ (in bold)
- Number of data l , dimension of input vector n , y_p (targets) $p=1..l$

d_p or t_p

$$\mathbf{w}^T \mathbf{x} + w_0 = w_0 + w_1 x_1 + w_2 x_2 + \dots + w_n x_n = w_0 + \sum_{i=1}^n w_i x_i \quad (\text{eq. 1})$$

- Note that sometimes (in NN) the transpose notation T in $\mathbf{w}^T$ is omitted
- w_0 is the *intercept, threshold, bias, offset*....

Often it is convenient to include the constant $x_0 = 1$ so that we can write eq.1 as :

$$\mathbf{w}^T \mathbf{x} = \boxed{\mathbf{x}^T \mathbf{w}}$$

Inner product

$$\mathbf{x}^T = [1, x_1, x_2, \dots, x_n]$$

$$\mathbf{w}^T = [w_0, w_1, w_2, \dots, w_n]$$

So, the "linear" model can be written
as a function that for each $\mathbf{x}_p$ compute:

$$h(\mathbf{x}_p) = \mathbf{x}_p^T \mathbf{w} = \sum_{i=0}^n x_{p,i} w_i$$

**$\mathbf{w}$: continuous (free) parameters:
"weights"**

Learning algorithm: just wait!



Dip. Informatica
University of Pisa

- For the learning algorithm in the multidimensional case for linear regression: please wait.
- We will provide it along with the learning algorithm of the linear classifier in the next few slides.

Summing up:

- Given the data set and the linear model, we can state the learning problem as **LMS problem**
- Once we find the best w parameters values, we have our $h_w(x)$ for regression purposes
- For students that need a soft intro to just LMS Regression:
 - <https://svivek.com/teaching/machine-learning/lectures/slides/linear-models/lms-regression.pdf>

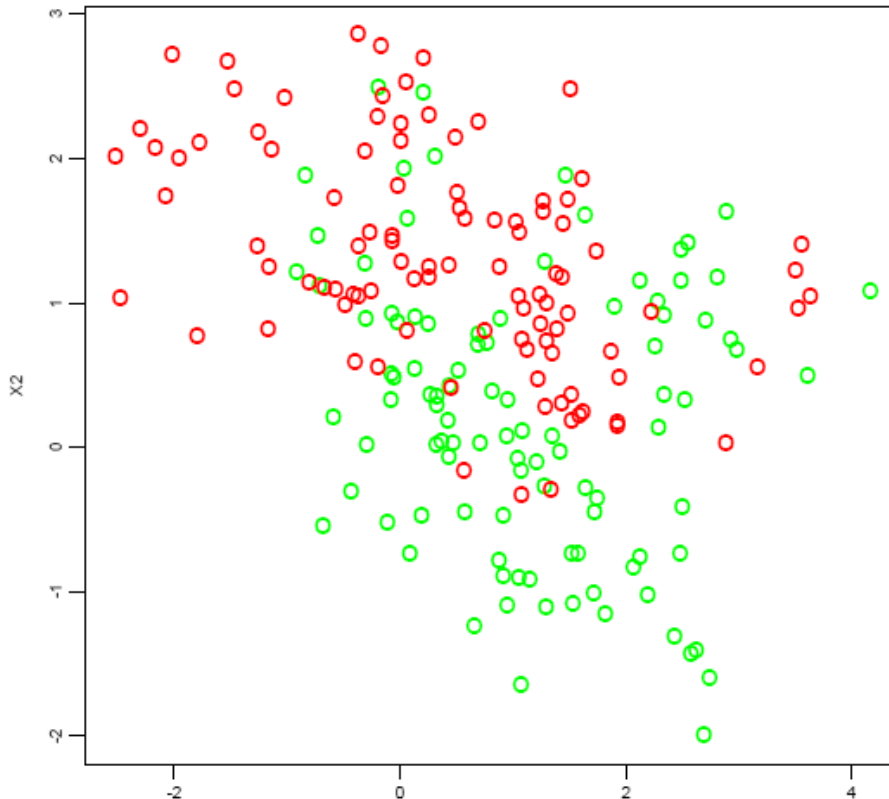
Classification

What we are looking at

1. The classification by **hyperplanes**
2. See how the model works after training (“**use it**” slides)
3. How to state/formulate a **(regression)/classification learning problem** for a linear model by LMS
4. How to **derive the learning algorithm**
5. Proposing **two learning algorithms** to build a linear classifier

Problem: example

Raw Data with a Binary Response



200 points generated in $\mathbb{R}^2$ from an unknown distribution; 100 in each of two classes.

Can we build a rule to predict the color of future points?

Data may be generated by gaussian distribution (for each class) with different means
or by a mixture of different low variance gaussian distributions.

We reuse the linear model

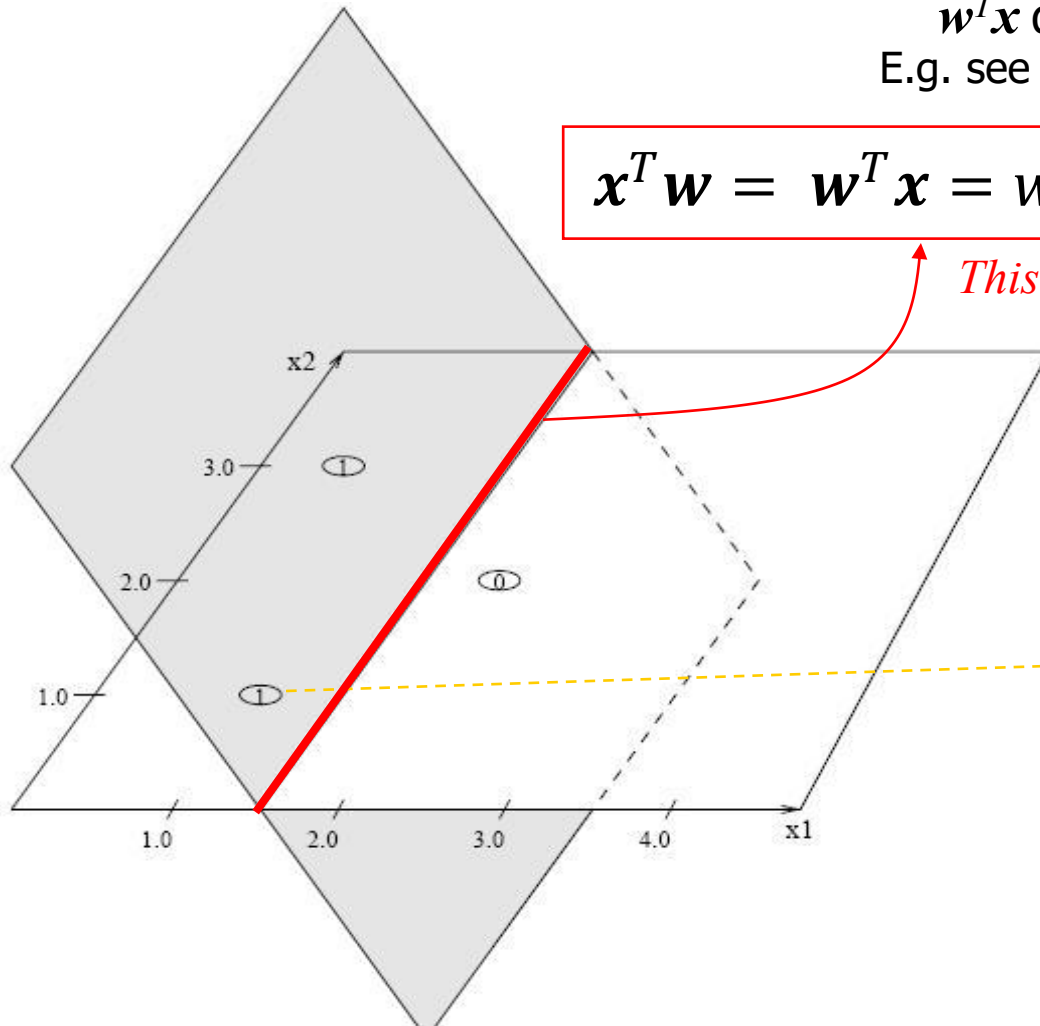
- The same models (used for regression) can be used for **classification**: categorical targets (y or d), e.g. **0/1** or **-1/+1**.
- In this case we use a **hyperplane** ($w^T x$) assuming negative or positive values
- We exploit such models to decide if a point x belong to positive or negative zone of the hyperplane (to classify it)
- So we want to set w (by learning) s.t. we get good classification accuracy

Geometrical view: **hyperplane**

$w^T x$ define a hyperplane.
E.g. see the picture for 2 variables

$$x^T w = w^T x = w_0 + w_1 x_1 + w_2 x_2 = 0$$

This defines the decision boundary



Can be used to classify:

Examples $\langle (x_1, x_2), y \rangle$:

$\langle (1.0, 1.0), 1 \rangle$

$\langle (0.5, 3.0), 1 \rangle$

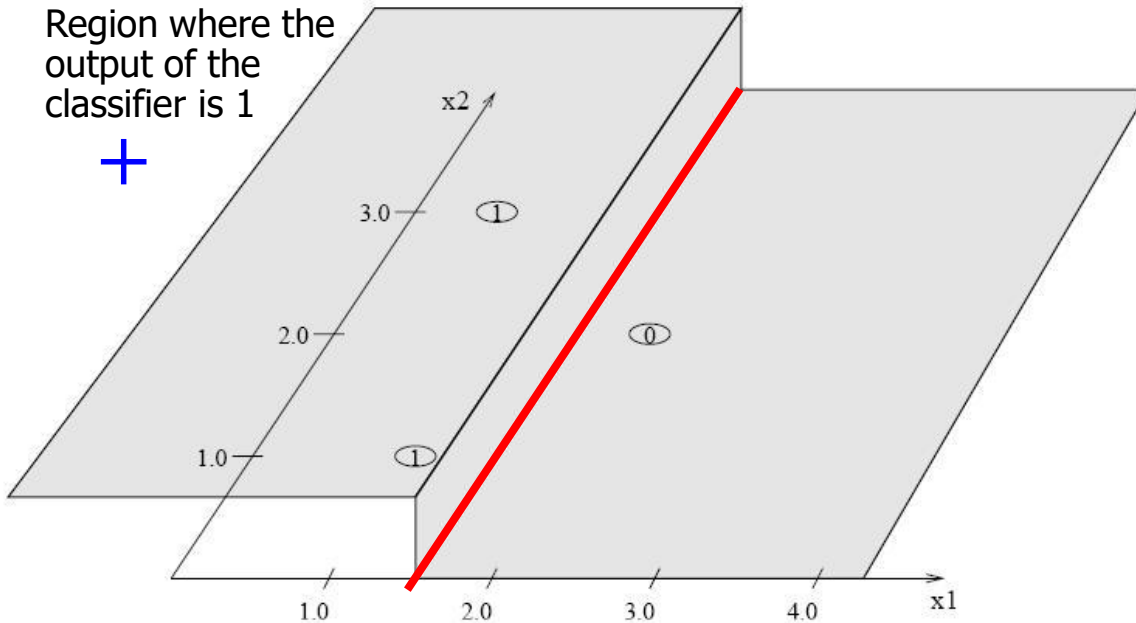
$\langle (2.0, 2.0), 0 \rangle$

Geometrical view: classifier

Introducing a threshold function

Region where the
output of the
classifier is 1

+



Using $\mathbf{x}_p$ and
including w_0 in $\mathbf{w}$

In this slide $\mathbf{w}$ is omitted from $h_{\mathbf{w}}$ (we use just h)

Examples:

$\langle (1.0, 1.0), 1 \rangle$

$\langle (0.5, 3.0), 1 \rangle$

$\langle (2.0, 2.0), 0 \rangle$

$[0,1]$ output range

$$h(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{w}^T \mathbf{x} + w_0 \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

or

$[-1,+1]$ output range

$$h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + w_0)$$

$$h(\mathbf{x}_p) = \text{sign}(\mathbf{x}_p^T \mathbf{w}) = \text{sign}\left(\sum_{i=0}^n x_{p,i} w_i\right)$$

Classification by linear decision boundary [repetita]

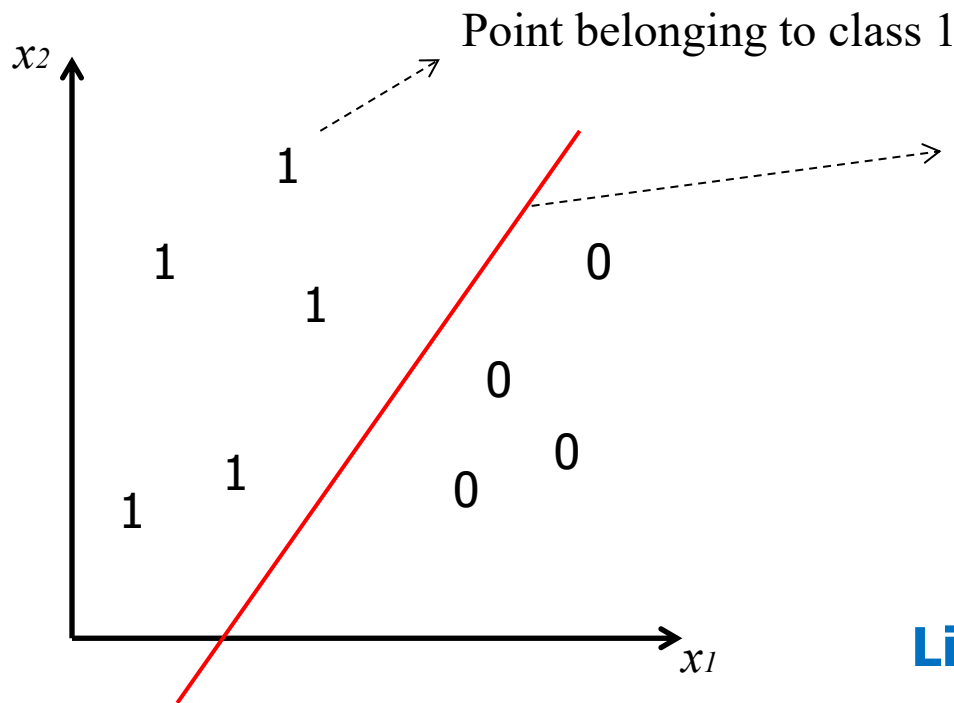


Dip. Informatica
University of Pisa

The classification may be viewed as the allocation of the input space in decision regions (e.g. **0/1**)

Example: linear separator on

2-dim instance space $\mathbf{x}=(x_1, x_2)$ in $\mathbb{R}^2$, $f(\mathbf{x})=0/1$ (or $-1/+1$)



Separating (hyper)plane
Decision boundary: $\mathbf{x}$ s.t.

$$\mathbf{w}^T \mathbf{x} + w_0 = w_1 x_1 + w_2 x_2 + w_0 = 0$$

$$h(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{w}^T \mathbf{x} + w_0 \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

[0,1]
output range

or

$$h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + w_0)$$

[-1,+1]
output range

Linear threshold unit (LTU)

How many? (H): set of dichotomies induced by hyperplanes

Threshold (bias w_0)

Note that, given the bias w_0 , in the LTU

saying $h(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + w_0 \geq 0$

is equivalent to say $\mathbf{w}^T \mathbf{x} \geq -w_0$

with $-w_0$ as the «threshold» value

- The two forms identify the same positive zone of the classifier
- The second one emphasizes the role of the bias as a threshold value to “activate” the +1 output of the classifier.

Use it: Example (AIMA)

Classify a new data (x_1, x_2)

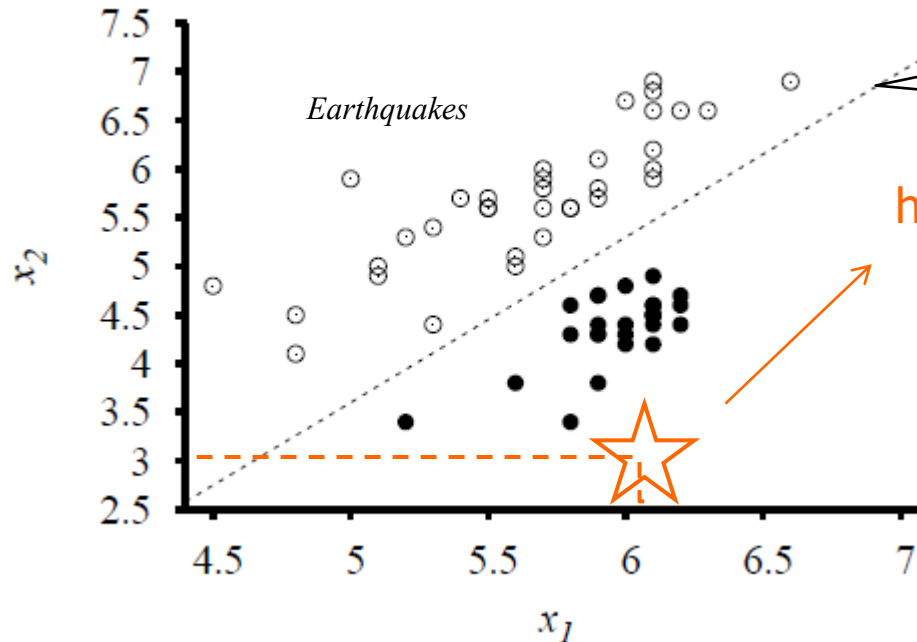
Find h s.t. given (x_1, x_2) return 0/-1 for *Earthquakes* and 1 for *Nuclear Explosion*

Some alg. finds this
decision boundary:

$$w_0 + w_1x_1 + w_2x_2 = 0$$



$$-4.9 + 1.7x_1 - 1x_2 = 0$$



Seismic data.

x_1 body wave magnitude, x_2 surface wave magnitude

Earthquakes (white) Nuclear Explosion (black) 1982-1990 Asia

hence $h(6,3) = \text{sign}(w_0 + w_1 * 6 + w_2 * 3) =$
 $= \text{sign}(-4.9 + 1.7 * 6 - 1 * 3)$
 $= \text{sign}(2.3) = +1 \rightarrow \text{nuclear expl.}$



Getting new examples is expensive;-)

Use it: Example (Spam)

- Find $h(mail) + 1$ for *spam*, -1 *not-spam*
 - Features $\Phi(mail) = \text{words } [0/1] \text{ or phrases ("free money")} [0/1] \text{ or length [integer]}$
 - e.g. $\phi_k(\mathbf{x}) = \text{contain}(\text{word}_k)$ [*bag of words* representation]
- $\mathbf{w} \rightarrow$ weight contribution of the input features to prediction
 - e.g. positive weight for "free money", negative for ".edu" or "unipi"
- $\mathbf{x}^T \mathbf{w}$ is the weight combination
- $h_w(x)$ provides the threshold to decide spam/not spam

$$h_w(\mathbf{x}) = \text{sign}\left(\sum_k w_k \phi_k(\mathbf{x})\right) > 0 \rightarrow +1 = \text{Spam} !$$

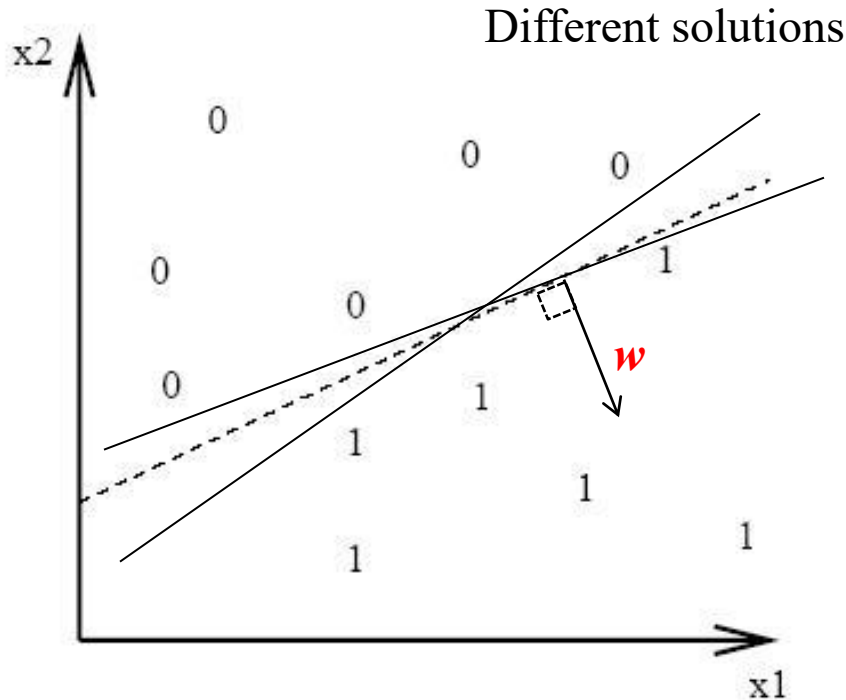


Some useful properties

(we will use them in future lectures)

$$\mathbf{w}^T \mathbf{x} = w_1 x_1 + w_2 x_2 + w_0 = 0$$

Exercise: Draw $x_2 = -x_1 w_1/w_2 - w_0/w_2$ with different $\mathbf{w}$ values



A linearly separable problem

- If $w_0=0$ the line goes through the *origin* of the coordinate system.
- If $n > 2 \rightarrow$ *hyperplane* (decision boundary)
- *Scaling freedom*: the same decision boundary multiplying $\mathbf{w}$ by K
- $\mathbf{w}$ is a vector *orthogonal* to the hyperplane:

Given $\mathbf{x}_a, \mathbf{x}_b$ (belonging to the sep. hyperplane):
 $\mathbf{w}^T \mathbf{x}_a + w_0 = 0$; $\mathbf{w}^T \mathbf{x}_b + w_0 = 0$ (take the diff., still $=0$) $\rightarrow$
 $\mathbf{w}^T (\mathbf{x}_a - \mathbf{x}_b) = 0$ (dot prod. $=0$) $\rightarrow$ *orthogonal vectors*

- If it exist, there are many possible hyperplanes separating these points:
also many ML alg.!!!

Learning

Learning Algorithms

- We are going to introduce 2 learning algorithms for the regression and for the classification task using a linear model, both based on LMS:
 1. A direct approach based on **normal equation** solution
 2. An iterative approach based on **gradient descent**
- We start **redefining the learning problem** and the loss for them (for l data and multidimensional inputs)

The learning problem (classification tasks)



Dip. Informatica
University of Pisa

- **Given** a set of l training examples $(\mathbf{x}_p, y_p)$ and a loss function (measure) L

$$y_p = \{0, 1\} \text{ or } y_p = \{-1, +1\}$$

- **Find:** The weight vector $\mathbf{w}$ that minimizes the expected loss on the training data

$$R_{emp} = \frac{1}{l} \sum_{p=1}^l L(h(\mathbf{x}_p), y_p)$$

- For classification: Using a piecewise constant (over $\text{sign}(\mathbf{w}^T \mathbf{x})$) for the loss can make this a difficult problem.
- Assume we still use the *least squares (as for the regression case)*

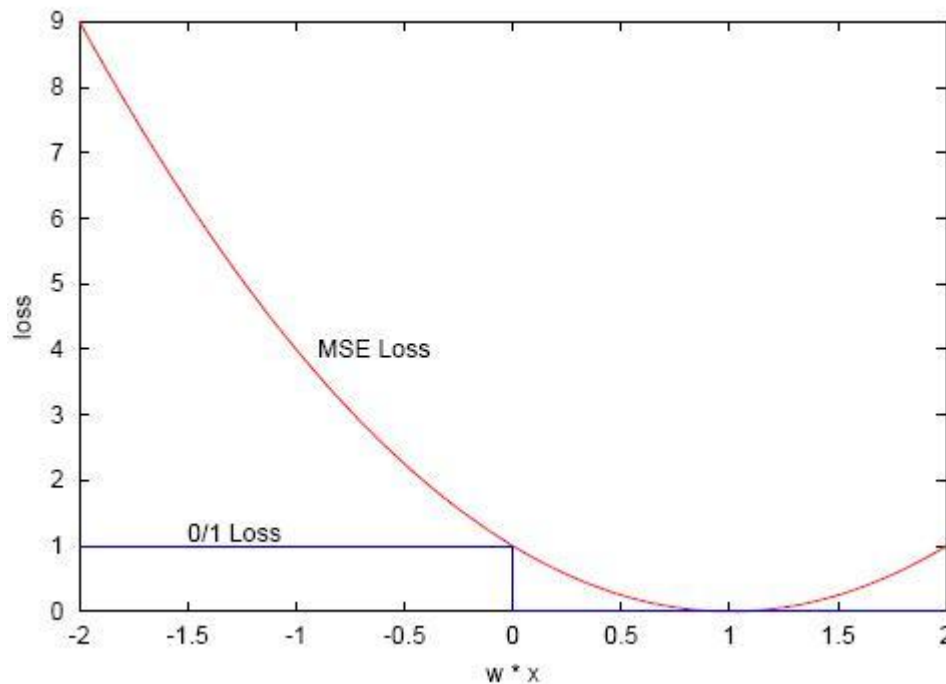
$$E(\mathbf{w}) \propto \sum_{p=1}^l (y_p - \mathbf{x}_p^T \mathbf{w})^2 = \sum_{p=1}^l (y_p - \mathbf{w}^T \mathbf{x}_p)^2$$

Approximating expected loss by a smooth function (#)

- Initially, we can make the optimization problem easier by replacing the original objective function L (*0/1 loss*) by a **smooth, differentiable function**. For example, consider the popular *mean squared error (MSE loss)*:

Both losses satisfy the minimization of error (*),

Let us start with LMS avoiding to introduce combinatorial problems



(*) Example: y (target)=1,
 $h(\mathbf{x})=1$ if $\mathbf{w}^T \mathbf{x} > 0$,
 $\rightarrow$ No err. when $\mathbf{w}^T \mathbf{x} > 0$
for the classifier

Hence no classif. error
minimizing either

- 0/1 loss
- or MSE loss

(solution is for both
on the right part in the plot)

Learning (a classifier) by Least Squares



Dip. Informatica
University of Pisa

- Find optimal values for $\mathbf{w}$ (for fitting of training (TR) data) by *least squares*:
- Given** a set of l training examples $(\mathbf{x}_p, y_p)$, **find** $\mathbf{w}$ to *minimize* the residual sum of squares:

$$E(\mathbf{w}) = \sum_{p=1}^l (y_p - \mathbf{x}_p^T \mathbf{w})^2 = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$$

Where $\mathbf{x}_p$ is p -th input vector, y_p the target for p , $\mathbf{w}$ free par., l num. of examples, n input dim.

Min error: if $y_p = 1$ then $\mathbf{x}_p^T \mathbf{w}$ go toward 1 $\rightarrow$ no class. error
 if $y_p = -1$ then $\mathbf{x}_p^T \mathbf{w}$ go toward -1 $\rightarrow$ no class. error

Note: in $E(\mathbf{w})$ we do **not** use $h(\mathbf{x})$, as for regression, to hold a continuous differentiable loss (because $h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x})$ for classification) !!!

- This is a quadratic function $\rightarrow$ minimum always exists (but may be not unique) [see course of CM or O4DS (@DSBI)]
- $\mathbf{X}$ is a matrix $l \times n$ with a row for each input vector $\mathbf{x}_p$
- Note: The same approach is used for a *regression problem*

Learning Algorithms

- We will introduce 2 learning algorithms for the regression and for the classification tasks using a linear model, both based on LMS

1. A direct approach based on **normal equation** solution

2. An iterative approach based on **gradient descent**



Normal equation & direct approach solution

- Differentiate $E(\mathbf{w})$ with respect to $\mathbf{w}$:
Blackboard or Exercise (a next slide).

Result synthesis:

- In the derivation we find that
$$\frac{\partial E(\mathbf{w})}{\partial w_j} = -2 \sum_{p=1}^l (y_p - \mathbf{x}_p^T \mathbf{w}) x_{p,j}$$

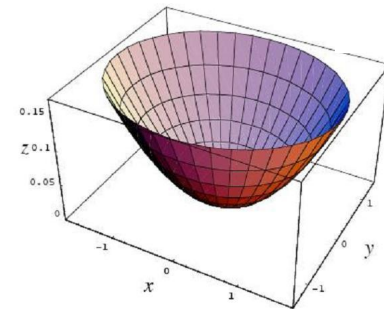
- We can get the **normal equation**
(point with gradient of E w.r.t $\mathbf{w} = 0$):

$$(\mathbf{X}^T \mathbf{X}) \mathbf{w} = \mathbf{X}^T \mathbf{y}$$

- If $\mathbf{X}^T \mathbf{X}$ is not singular the unique solution is given by

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = \mathbf{X}^+ \mathbf{y} \quad \text{`+' Moore-Penrose pseudoinverse (also if } \mathbf{X} \text{ is not invertible)}$$

- Else the solution are infinite (satisfying the normal equation):
we can choose the *min norm* ($\mathbf{w}$) solution



See also
CM course

Direct approach by SVD

- The *Singular Value Decomposition (SVD)* can be used for computing the pseudoinverse of a matrix (X^+)

$$X = U \underset{\substack{\downarrow \\ \text{diagonal}}}{\Sigma} V^T \Rightarrow X^+ = V \underset{\substack{\text{by replacing every nonzero entry by its reciprocal}}}{\Sigma^+} U^T$$

- Moreover we can apply directly SVD to compute $w = X^+ y$ obtaining the minimal norm (on w) solution of least squares problem.
- Note:** This is the **learning alg.** for the direct approach solution on w (or say in *closed form*)
- A practical tool. E.g. see “numerical recipes” in C and many numerical/statistical tools and scientific library (also in R, Octave, Matlab,...)
 - e.g. ARMADILLO: since 2011/12 C++ linear algebra library, NumPy etc.
- Many algorithms addressing the problems of efficiency and stability
- See also **CM** course (may be later) or O4DS (@DSBI)

Solution (to find the normal eq.)

Make as an Exercise !!!(#)

$$\begin{aligned}
 \frac{\partial E(\mathbf{w})}{\partial w_j} &= \frac{\partial \sum_{p=1}^l (y_p - \mathbf{x}_p^T \mathbf{w})^2}{\partial w_j} = \sum_{p=1}^l 2(y_p - \mathbf{x}_p^T \mathbf{w}) \frac{\partial (y_p - \mathbf{x}_p^T \mathbf{w})}{\partial w_j} = \\
 &= \sum_{p=1}^l 2(y_p - \mathbf{x}_p^T \mathbf{w}) \left(0 - \cancel{\frac{\partial (x_p)_1 w_1}{\partial w_j}} - \cancel{\frac{\partial (x_p)_2 w_2}{\partial w_j}} - \dots - \cancel{\frac{\partial (x_p)_j w_j}{\partial w_j}} - \dots \right) = \\
 &= \sum_{p=1}^l 2(y_p - \mathbf{x}_p^T \mathbf{w}) \left(-\frac{\partial (x_p)_j w_j}{\partial w_j} \right) = -2 \sum_{p=1}^l (y_p - \mathbf{x}_p^T \mathbf{w}) (x_p)_j
 \end{aligned}$$

Only the component j is not 0

Imposing this =0, we can easily obtain the *normal equation* (first by “sums”, then in matrix notations)

And we also obtained the gradient of E

rewritten as: $\frac{\partial E(\mathbf{w})}{\partial w_j} = -2 \sum_{p=1}^l \left((y_p - \mathbf{x}_p^T \mathbf{w}) \right) x_{p,j} = -2 \sum_{p=1}^l \delta_p x_{p,j}$

(Note: In the original image, δ_p is written in blue and the term $(y_p - \mathbf{x}_p^T \mathbf{w})$ is circled with a dashed line.)

(we will use this form in the future, with Neural Networks)

Other approaches to LS

Many, for instances by an **iterative/gradient descent technique** we can search for:

- More efficient solutions (previous is cubic with dim of matrix X)
- Regularization (to reduce complexity of the model)
- Better approximation with noisy data by stopping searching before the minimum
- Other approaches that can be applied also to NON-LINEAR models !!!

Hence, we are going to present the second (more important) learning algorithm

Learning Algorithms

- We will introduce 2 learning algorithms for the regression and for the classification tasks using a linear model, both based on LMS
 1. A direct approach based on **normal equation** solution
 2. An iterative approach based on **gradient descent**

This approach will be the basis for fundamental approaches we will see later

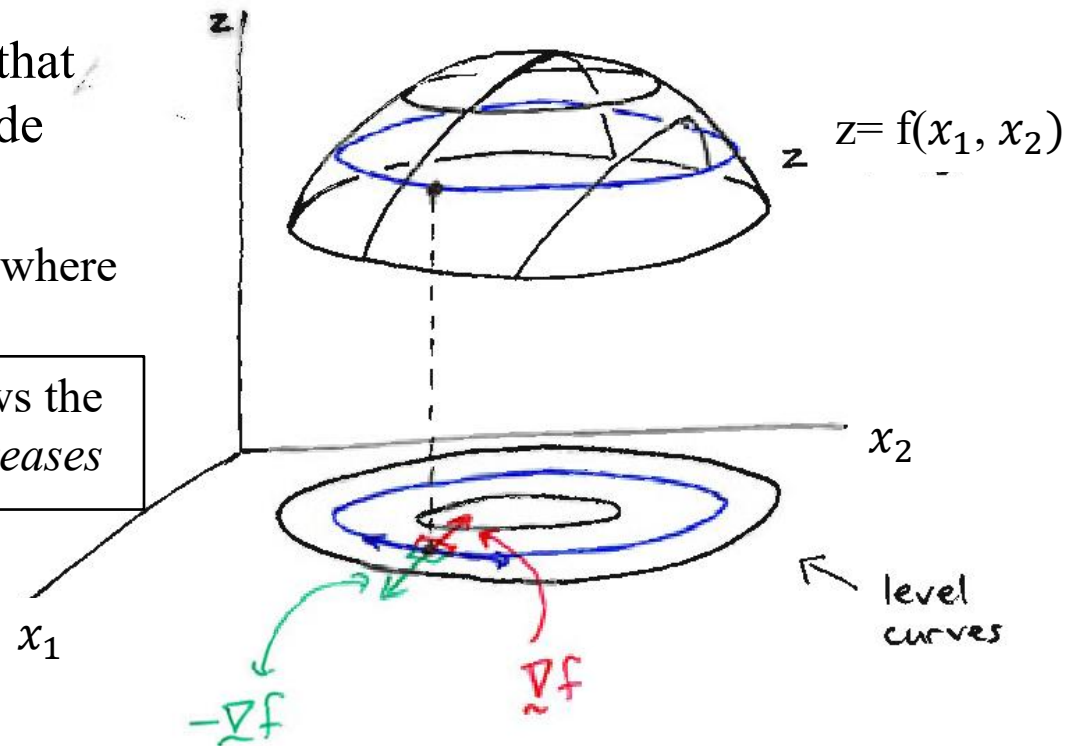


REPETITA

Gradient on a surface

- The gradient at a point is a vector pointing in the **direction of the steepest slope** at that point.
- The steepness of the slope at that point is given by the magnitude of the gradient vector
 - The **gradient** shows the direction where the function grows

– The **negative of the gradient** shows the direction where the function *decreases*



Gradient descent

- Previous derivation suggest the line to construct an **iterative algorithm** based on:

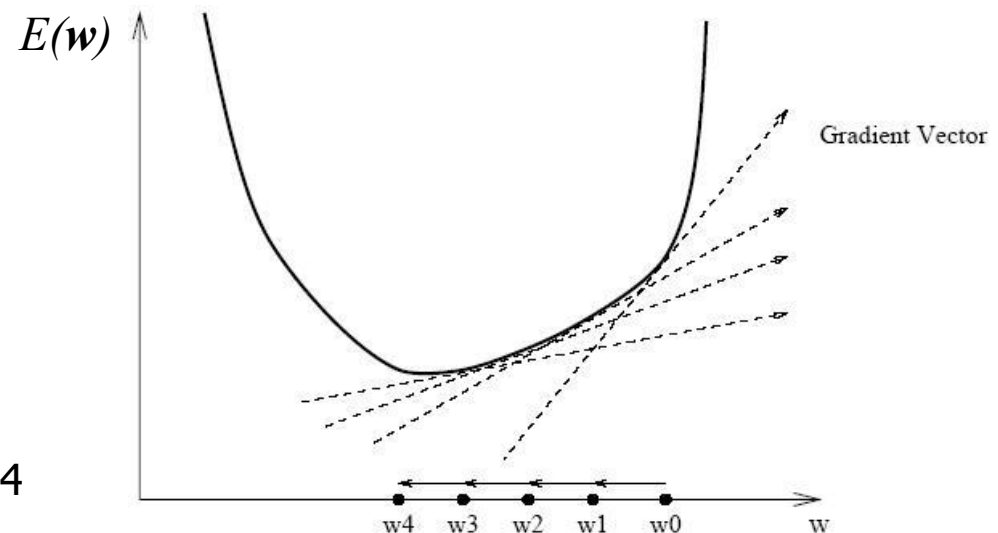
$$\frac{\partial E(\mathbf{w})}{\partial w_j} = -2 \sum_{p=1}^l (y_p - \mathbf{x}_p^T \mathbf{w})(\mathbf{x}_p)_j$$

Where $\mathbf{x}_p$ is p -th input pattern,
 y_p the target for p , $\mathbf{w}$ free par.,
 l num. of examples

→ Component j of pattern p , also $x_{p,j}$

- Gradient** = *ascent direction*: we can move toward the minimum with a **gradient descent** (changing $\mathbf{w}$ with $\Delta \mathbf{w} = -$ gradient of $E(\mathbf{w})$)

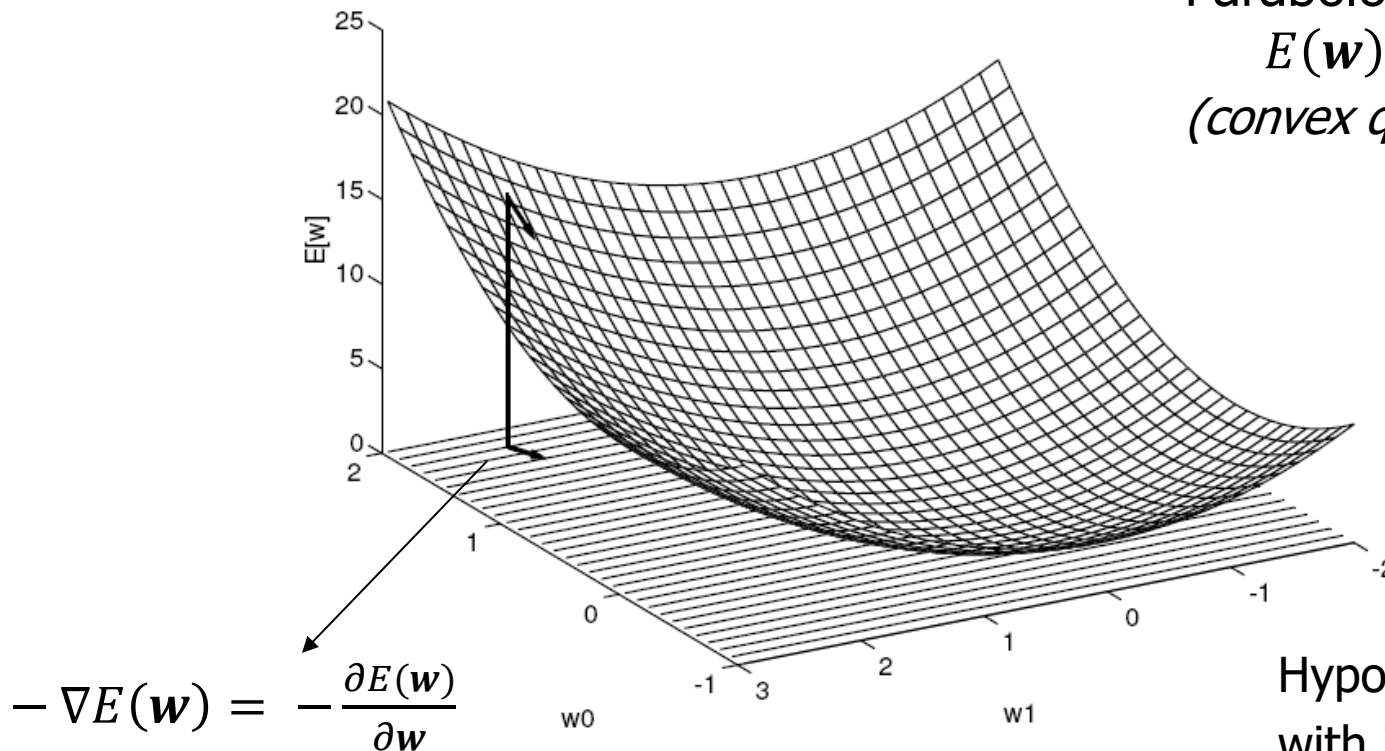
- *Local search*: it begins with an initial weight vector. We modify it iteratively to decrease up to minimize the error function (steepest descent).



A single w at step 0,1,2,3,4

Error surface for linear model with 2 weights ($\mathbf{w}$)

Paraboloid for the
 $E(\mathbf{w}) = E([w_0, w_1]^T)$
(convex quadratic function)



Hypothesis space
with 2 parameters
 w_0, w_1

Our "compass" to find the minimum

$$\Delta w_j$$

$$\Delta w_j = -\frac{\partial E(\mathbf{w})}{\partial w_j} = 2 \sum_{p=1}^l (y_p - \mathbf{x}_p^T \mathbf{w}) x_{p,j}$$

$\downarrow$ $\downarrow$

*2 is constant that can be
ignored to develop the
algorithm*

Component i of pattern p

The gradient vector

$$\Delta \mathbf{w} = -\nabla E(\mathbf{w}) = -\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} = \begin{bmatrix} -\frac{\partial E(\mathbf{w})}{\partial w_0} \\ -\frac{\partial E(\mathbf{w})}{\partial w_1} \\ -\frac{\partial E(\mathbf{w})}{\partial w_2} \\ -\frac{\partial E(\mathbf{w})}{\partial w_j} \\ \dots \\ -\frac{\partial E(\mathbf{w})}{\partial w_n} \end{bmatrix} = \begin{bmatrix} \Delta w_0 \\ \Delta w_1 \\ \Delta w_2 \\ \Delta w_j \\ \dots \\ \Delta w_n \end{bmatrix}$$

We can work in a multi-dim space without the need to visualize it

Using the Delta Rule

- Hence, as **iterative approach** we will move using a learning rule based on a «delta» (changing) of w proportional to the (opposite) of the local gradient
- The «movements» will be made iteratively according to

$$w_{\text{new}} = w + \eta \Delta w$$

(or component-wise, i.e. for each w_j)

- that is the “**learning rule**”
- and *eta* (η) is the “step size” (learning rate) parameter (ruling the speed of our gradient descending)

Gradient descent algorithm

A simple (fundamental) algorithm:

1) Start with weight vector $\mathbf{w}_{\text{initial}}$ (small), fix η (*eta*) ($0 < \eta < 1$).

2) Compute $\Delta \mathbf{w} = -\text{"gradient of } E(\mathbf{w}) \text{"} = - \frac{\partial E(\mathbf{w})}{\partial \mathbf{w}}$ (or for each w_j)

3) Compute $\mathbf{w}_{\text{new}} = \mathbf{w} + \eta \Delta \mathbf{w}$ (or for each w_j)

Repeat 2) 3) until convergence or $E(\mathbf{w})$ is "sufficiently small"

- $\Delta \mathbf{w}/l$: *least mean squares* (dividing by l , that will be the standard case)

- Batch versions ($\Delta \mathbf{w}$ after each "epoch" of l training patterns)

- *eta* (η): step size = *learning rate*: speed/stability trade-off: it can be (gradually) decreased to zero (guarantee convergence, avoiding oscillation around the min.): many variants will be introduced later



Batch/On-line

- For **batch version** the gradient is the sum over all the l patterns:

$$\Delta w_j = -\frac{\partial E(\mathbf{w})}{\partial w_j} = 2 \sum_{p=1}^l (y_p - \mathbf{x}_p^T \mathbf{w}) x_{p,j}$$

- provide a more “precise” evaluation of the gradient over a set of l data

And we upgrade the weights after this sum

- For the **on-line/stochastic version** we upgrade the weights with the error that is computed for each pattern
 - hence, the 2nd pattern output is based on weights already updated from the 1st, and so ahead
 - It makes progress with each examples it looks at: it can be the faster, but need smaller *eta*:

$$\Delta_p w_j = -\frac{\partial E_p(\mathbf{w})}{\partial w_j} = 2(y_p - \mathbf{x}_p^T \mathbf{w}) x_{p,j}$$

- We will see intermediate cases later (as **mini-batch**)



Examples

Batch algorithm

We update $\mathbf{w}$ after (repeating) an "epoch" of l training data $\rightarrow$ (*blue curve*)

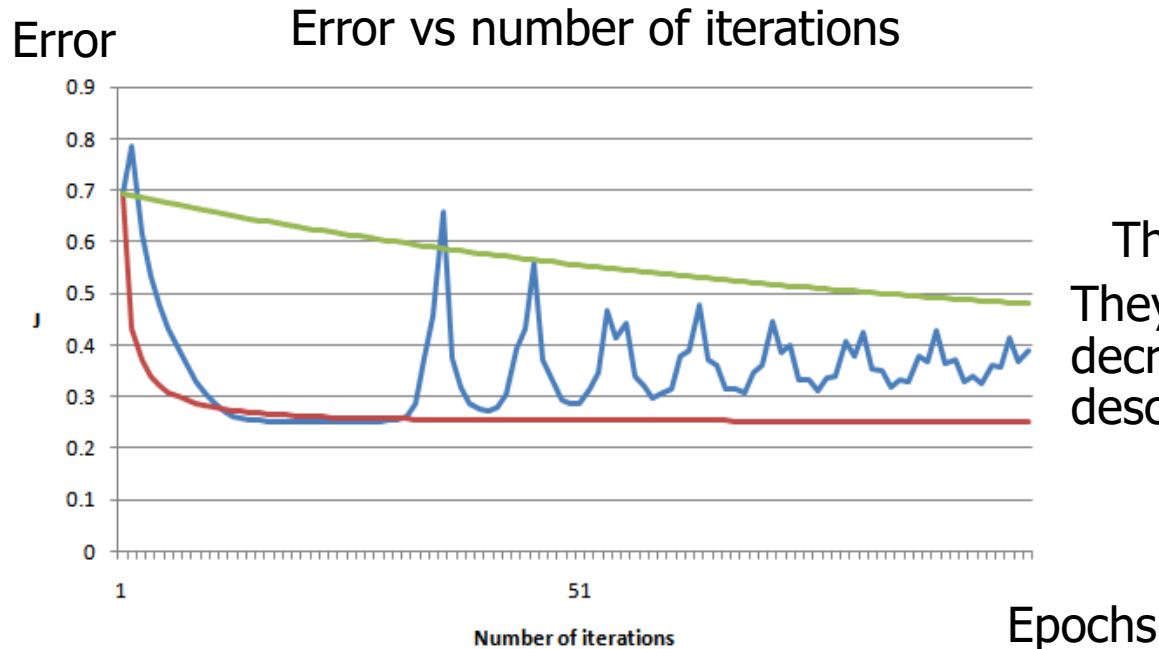
On-line algorithm (stochastic gradient descent - *SGD*)

We update $\mathbf{w}$ after each pattern p by $\Delta_p \mathbf{w}$ for each pattern (*purple and green curves*)



Paths over the error surface by Batch or On-line version

Learning curve examples



These are **learning curves**:
They show how the error decreases through gradient descent iterations

P.S. No relation with color in the previous slide!

Exercise: *1 is slow, 1 is unstable, 1 is good: which one?*
Which with high or low eta value?

Gradient descent as Error correction delta rule

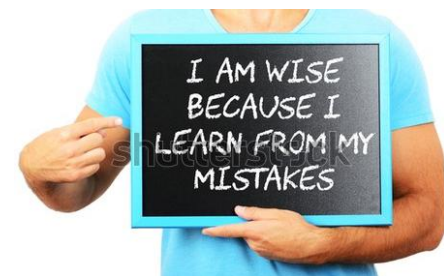
$$\Delta w_j = 2 \sum_{p=1}^l (y_p - x_p^T \mathbf{w}) x_{p,j}$$

$\mathbf{w}_{\text{new}} = \mathbf{w} + \eta \Delta \mathbf{w}$

Where $x_{p,j}$ is the component j of the input pattern p , y_p the target for p , $\mathbf{w}$ free par., l num. of examples.

The constant 2 can be omitted

- This is an "error correction" rule (Widrow-Hoff or **delta rule**) that change each w_j proportionally to the error (target y - output):
 - E.g. (target y – output) = err=0 $\rightarrow$ no correction
 - (input $_j$ >0) if err + (output is too low), positive delta $\rightarrow$ increase $w_j \rightarrow$ increment the output $\rightarrow$ less err
 - (input $_j$ >0) if err - (output is too high), negative delta $\rightarrow$ decrease $w_j \rightarrow$ reduce output $\rightarrow$ less err
 - ... [exercise: all the cases]
- We improve by learning from previous errors
 - "seeking and blundering we learn (Goethe)"



DeltaW as Error Correction Learning rule: Example



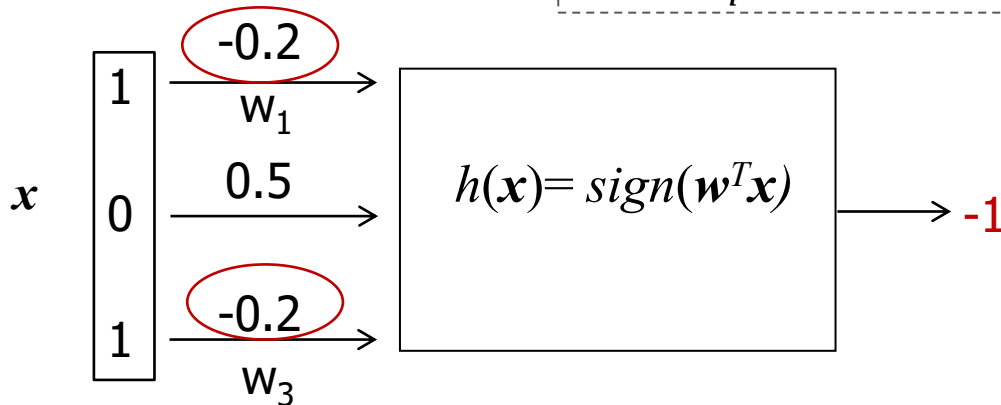
Dip. Informatica
University of Pisa

$(w_0=0)$

$$\Delta w_j = \sum_{p=1}^l \cancel{\times} (y_p - \mathbf{x}_p^T \mathbf{w}) x_{p,j}$$

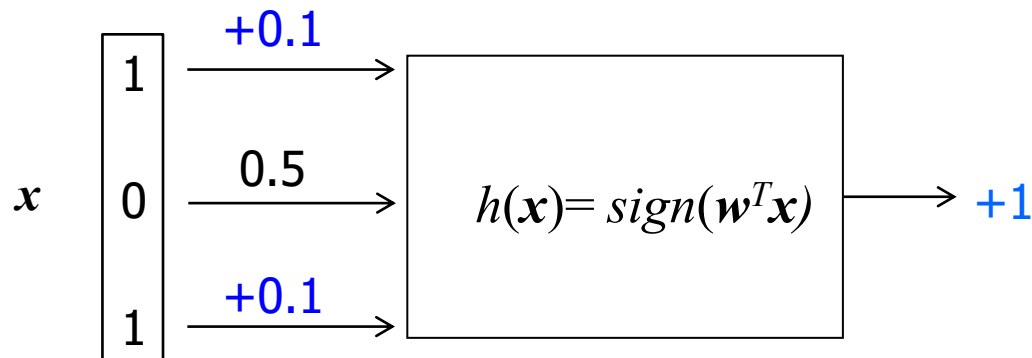
$$\mathbf{w}_{\text{new}} = \mathbf{w} + \eta \Delta \mathbf{w}$$

Inputs Weights



If **misclassified** (because target is +1) $\rightarrow$ compute $(1 - ???)$ $\rightarrow$ High positive delta for w_1 and w_3 $\rightarrow$ increase them proportionally (with η) to the *delta*, hence a positive value in this case [error correction rule] !!!

e.g. (see the figure):



Now is **correct** !!!

Exercise: compute the values for delta (hint: see eq. above) and for also η (eta)

Gradient descent: final discussion



Dip. Informatica
University of Pisa

Gradient descent approach it is simple and effective *local search* approach to LMS solution and

- It allows us to search through an *infinite (continuous) hypothesis* space!
- It can be easily always applied for *continues H* and *differentiable loss*
- NOT ONLY to linear models !!!! (we will see for Neural Networks and deep learning models)
- *Efficient?* Many improvements are possible, e.g. Newton & quasi-newton methods; Conjugate Gradient, ...! → [CM course*](#) (we will mention examples later, discussing Neural Networks)

Summarizing (Classification)

- Model **trained** (on TR set) with LS (LMS) on $\mathbf{w}^T \mathbf{x}$
 - by the simple gradient descent algorithm used for linear regression
- Model **used** for classification applying a threshold function, obtaining $h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x})$
- The error can be computed as *classification error* or number of misclassified patterns (not only by the Mean Square Error)

0/1 Loss

$$L(h(\mathbf{x}_p), d_p) = \begin{cases} 0 & \text{if } h(\mathbf{x}_p) = d_p \\ 1 & \text{otherwise} \end{cases}$$

$$\text{mean_err} = \frac{1}{l} \sum_{p=1}^l L(h(\mathbf{x}_p), d_p)$$

num_err

- ACCURACY** = mean of correctly classified = $(l - \text{num_err})/l$

Linear models: Discussion

Coming back to the problem: Linear model solution

Linear Regression of 0/1 Response

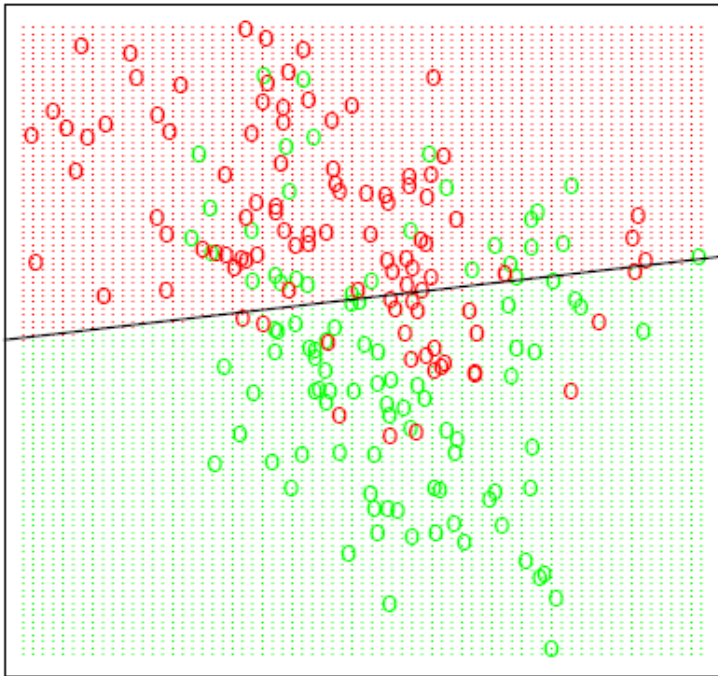


Figure 2.1 (© HTF 2001):

A classification example in two dimensions. The classes are coded as a binary variable GREEN = 0, RED = 1 and then fit by linear regression. The line is the decision boundary defined by $\mathbf{x}^T \mathbf{w} = 0.5$. The red shaded region denotes that part of input space classified as RED, while the green region is classified as GREEN.

$$h(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{x}^T \mathbf{w} > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Linear threshold unit

The *decision boundary* is $\{\mathbf{x} \mid \mathbf{x}^T \mathbf{w} = 0.5\}$ is linear (and seems to make many errors on the training data). Is it true?

Good or bad approximation?

(#)



Dip. Informatica
University of Pisa

- Possible scenarios (we know the true target function!)
 - Scenario 1: The data in each class are generated from a Gaussian distribution with uncorrelated components, same variances, and different means.
 - Scenario 2: The data in each class are generated from a mixture of 10 gaussians in each class.
- For Scenario 1, the linear regression rule (by LS) is almost **optimal** (is the best one can do). The region of overlap is inevitable (due to errors in the input data).
- For Scenario 2, it is far too rigid: next models for it!

* Least squares corresponds to the maximum likelihood criterion if the experimental errors have a normal distribution

Linear model (in ML): Inductive Bias (alla Mitchell)

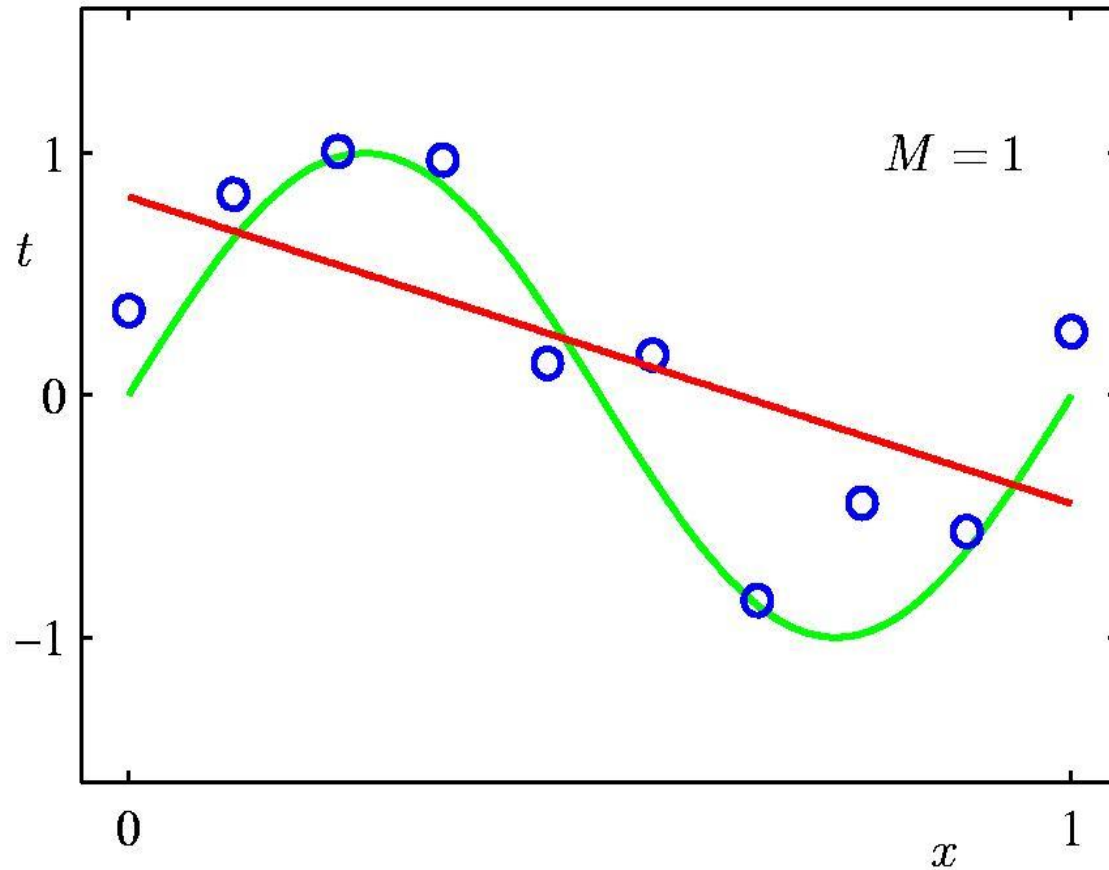


Dip. Informatica
University of Pisa

- **Language bias**: the H is a set of linear functions (may be very restrictive and rigid)
- **Search bias**: ordered search guided by the Least Squares minimization goal
 - For instance, we could prefer a different method to obtain a restriction on the values of parameters, achieving a different solutions with other properties (in particular to consider the generalization issue), ...

It shows that even for a “simple” model there are many possibilities.
We need a principled approach! (see theory of ML)...

Limitations: regression tasks for non linear problems



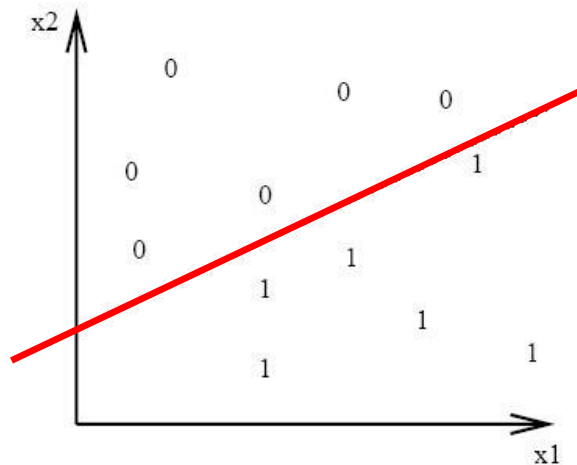
Too poor solution

Limitations (classification) (language bias)



Dip. Informatica
University of Pisa

- In geometry, two set of points in a two-dimensional plot are **linearly separable** when the two sets of points can be completely separated by a single line
- In general, two groups are *linearly separable* in n -dimensional space if they can be separated by an $(n - 1)$ -dimensional hyperplane.



- The linear decision boundary can provide exact solutions only for linearly separable sets of points

Example: Conjunctions

- We can represent conjunctions by the linear models, e.g.:
- Conjunctions** (see the example in the introduction lectures):

$$4 \text{ var.: } x_1 \wedge x_2 \wedge x_4 \leftrightarrow y$$

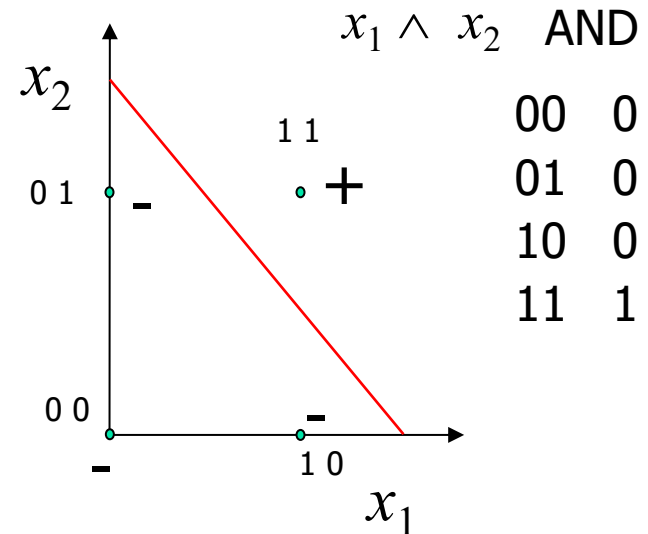
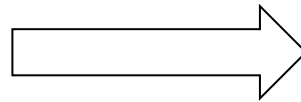
$$1 \ x_1 + 1 \ x_2 + 0 \ x_3 + 1 \ x_4 \geq 2.5$$

$$h(x) = \begin{cases} 1 & \text{if } w^T x + w_0 \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

In the plot:

$$2 \text{ var.: } x_1 \wedge x_2 \leftrightarrow y$$

$$1 \ x_1 + 1 \ x_2 \geq 1.5$$



w can be learned to find this solution

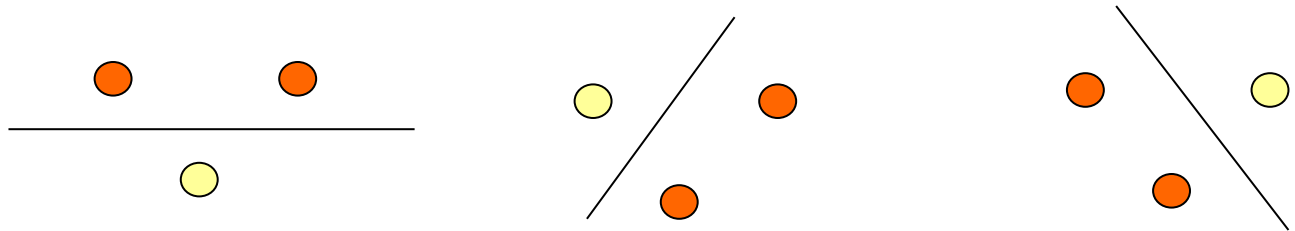
Limitations: classification tasks



Dip. Informatica
University of Pisa

- Given **3 points**, can we always find a separation plane for every assignment of $f(\mathbf{x})$?
No, 3 aligned points with 0 in the middle and others 1; yes if they are not aligned (existence!).

Here NOT complete:
There are 2^3 cases



- Given **4 points**, can we always find a separation plane for every assignment of $f(\mathbf{x})$?
No (XOR)

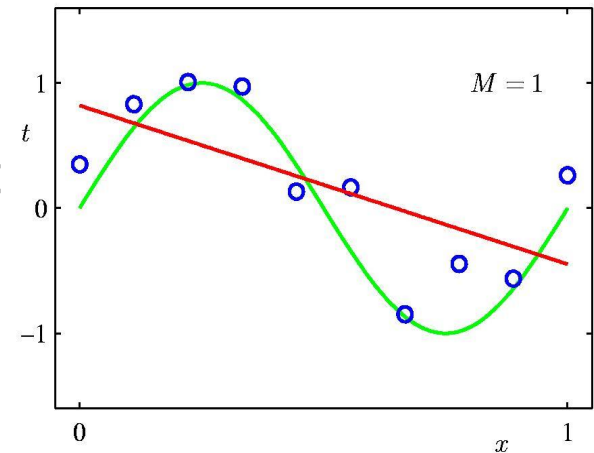
we can find a labeling such that the linear classifier fails to be perfect

		XOR
		00 0
		01 1
		10 1
		11 0

Exercise: try to redo it by yourself!

How to extend the linear model

- Note that in $h_{\mathbf{w}}(x) = w_1 x + w_0$ or $h_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^T \cdot \mathbf{x}$
- As Statistical Parametric models:
"linear" does not refer to this (red) straight line, but rather to the way in which the regression coefficients $\mathbf{w}$ occur in the regression equation



- Hence, we can use also transformed inputs, such as $x, x^2, x^3, x^4, \dots$ with *non-linear* relationship inputs and output, holding the learning machinery (Least Square solution) developed so far...

$$h_{\mathbf{w}}(x) = w_0 + w_1 x + w_2 x^2 + \dots + w_M x^M = \sum_{j=0}^M w_j x^j$$

polynomial regression

A generalization (**LBE**) (shown for regression*)



Dip. Informatica
University of Pisa

- Basis transformation: **linear basis expansion (LBE)**:

$$h_w(x) = \sum_{k=0}^K w_k \phi_k(x)$$

- Augment the input vector with additional variables which are transformations of x according to a function $\phi_k: R^n \rightarrow R$
- E.g.
 - Polynomial representation of x : $\phi(x) = x_j^2$ or $\phi(x) = x_j x_i$, or
 - Non-linear transformation of single inputs: $\phi(x) = \log(x_j)$, $\phi(x) = \text{root}(x_j)$,
 - Non-linear transformation of multiple input: $\phi(x) = ||x||$
 - *Splines*, ...,
- Typically: Number parameters $K > n$ (before it was n)
- The model is *linear in the parameters (also in ϕ , not in x): we can use the **same learning alg.** as before!*
- *Note: it can be applied for regression (here) or classification (Exercise: HOW?)*

Basis Expansion: examples



Dip. Informatica
University of Pisa

- Basis transformation: **linear basis expansion**:

$$h_{\mathbf{w}}(\mathbf{x}) = \sum_{k=0}^K w_k \phi_k(\mathbf{x})$$

EXAMPLES:

- [1-dim $\mathbf{x}$] $\phi_j(x) = x^j$.

$$h(\mathbf{x}) = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M = \sum_{j=0}^M w_jx^j$$

1-dim polynomial regression ($K=M$)

Already seen in
the introduction

- Or “any other”, e.g. $\phi(\mathbf{x}) = \phi([x_1, x_2, x_3])$

$$h(\mathbf{x}) = w_1x_1 + w_2x_2 + w_3\log(x_2) + w_4\log(x_3) + w_5(x_2x_3) + w_0$$

Basis Expansion criticism

- Which ϕ (ϕ) ? Toward the so called “**dictionary**” approaches
- **PROS**: Can model more complicated relationships (than linear) w.r.t. the inputs: it is more expressive.
- **CONS**: With *large* basis of functions, we easily risk *overfitting*, hence we require methods for controlling the complexity (*) of the model
 - *Curse of dimensionality* (the volume of the problem space increases so fast that the available data become sparse, density decreases, the data became no more sufficient to support the model complexity) → we will see later
 - Φ are *fixed before* observing training data
 - versus adaptive /non-linear in parameters e.g. NN!
- *We will see the alternative NN and SVM solutions!*

(*) Whereas **complexity** is **not** for the computational cost but a measure of the flexibility of the model to fit the data (see the VC-dim)

Introduction to Regularization

**Improvements:
How to control model complexity?**

Tikhonov regularization

Important !



Dip. Informatica
University of Pisa

- Many approaches... e.g. coefficient shrinkage:
- **Tikhonov regularization** (ridge regression):
 - possible to add constraints to the sum of values of $|w_j|$ penalizing models with high values of $|w|$ toward "sparse" models with less terms (weights $w_j=0$ or close to 0)
 - it means smoothed / less complex models

"Error" data term [(M)SE] Regularization/penalty term

(R_{emp} in the SLT)

$$Loss(\mathbf{w}) = \sum_{p=1}^l (y_p - \mathbf{x}_p^T \mathbf{w})^2 + \lambda \|\mathbf{w}\|^2$$

$$\sum w_j^2$$

Lambda (λ): regularization (hyper)parameter
(a small positive value chosen by the "model selection" phase)

The sum is over
the number of w

- Note that for the **objective function** we use here the name *Loss* (used for the model training cost function) to distinguish from the *Error* E (useful to evaluate the model error and used for the *data term* inside this Loss).
Hence, differently from previous assumptions the two terms are not more equivalent in our use (in the course).

AI view ;-)

Controlling Complexity (Tikhonov Regularization)

We redefine the goal: we do not want only a model that fits the data better, but the **simplest** model that fits the data better.



Data Fit (Training Error)

Pushes the model to fit the training data, in order to minimize the residuals.

λ

The Regularization Parameter

Decides how much weight to give to the penalty term.



Model Simplicity (Tikhonov Penalty)

Punishes large weights. Constrains the model to be smoother, simpler, and more generalizable.

Tikhonov Regularization: Solving it



Dip. Informatica
University of Pisa

$$Loss(\mathbf{w}) = \sum_{p=1}^l (y_p - \mathbf{x}_p^T \mathbf{w})^2 + \lambda ||\mathbf{w}||^2$$

- For the direct approach:

$$\mathbf{w} = \boxed{(\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1}} \mathbf{X}^T \mathbf{y} \quad \text{-----} \rightarrow \quad \text{this matrix is always invertible}$$

- For the gradient approach: we still apply –gradient of the Loss:
 - As an **Exercise** compute the gradient of the two terms (error and penalty terms) of the $Loss(\mathbf{w})$ w.r.t. weights w_i separately, using *eta* only for term E
 - Formulate the new update rule :
 - You will obtain:
$$\mathbf{w}_{\text{new}} = \mathbf{w} + \eta \Delta \mathbf{w} - 2 \lambda \mathbf{w}$$
 - That is a **weight decay** technique (basically add $2\lambda \mathbf{w}$ to the gradient)
 - E.g. with 0 gradient, it decreases the value of each w with a fraction of the old w

Andrej Nikolaevič Tikhonov



Russian mathematician
1906 –1993

Tikhonov regularization: trade-off



Dip. Informatica
University of Pisa

$$Loss(\mathbf{w}) = \sum_{p=1}^l (y_p - \mathbf{x}_p^T \mathbf{w})^2 + \lambda ||\mathbf{w}||^2$$

- Note the balancing (trade-off) between the two terms:
 - Small **lambda** (λ) value $\rightarrow$ minimizing the loss the focus is on obtaining a small error data term (first term, minimize just the training error) with a too complex model (high norm of the weights), the risk is of **overfitting**,
 - High **lambda** (λ) $\rightarrow$ minimizing the loss the focus is on the second term, hence the data error (first term) could grow too much, i.e. moving to **underfitting**
 - The trade-off is ruled by the value of **lambda** (λ)
 - We will see soon some examples
- The main advantage is that we have a concrete realization of the control of model complexity, easy to be implemented and of general applicability.

Tikhonov and SLT

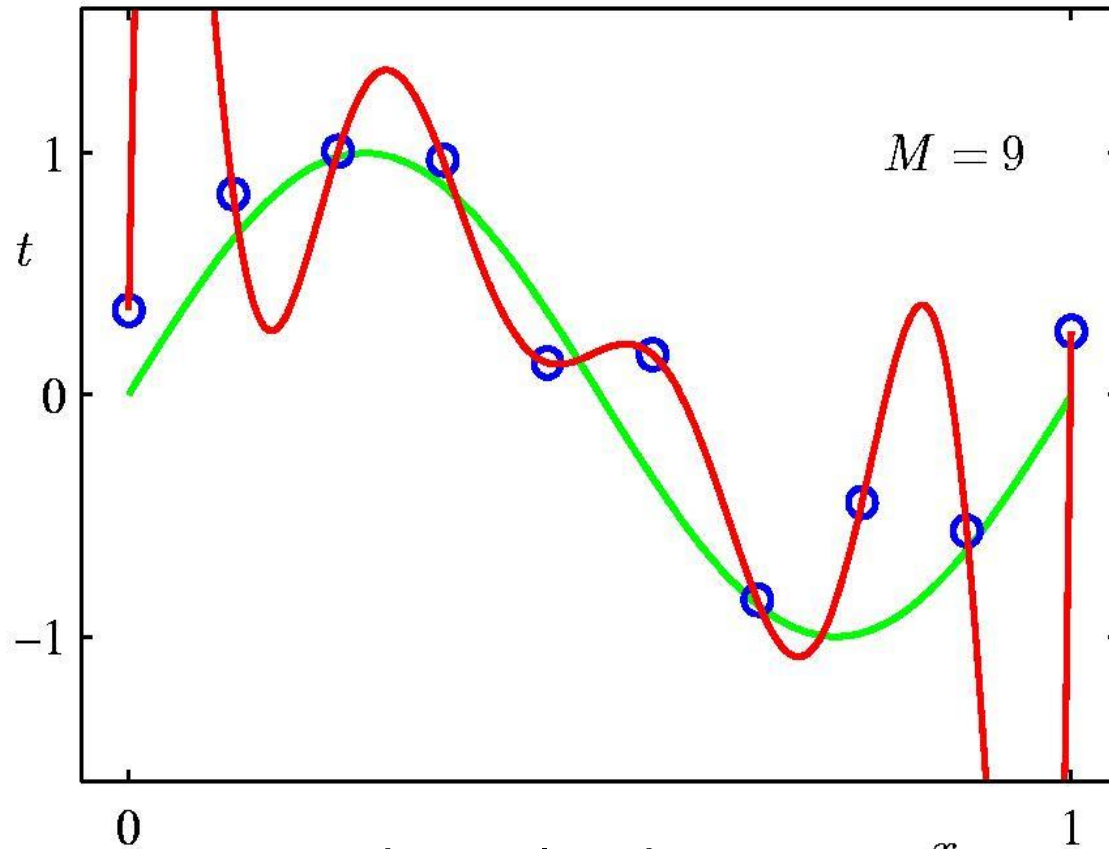
- The penalty term **penalizes high value of the weights** and tends to drive all the weights to smaller values
 - E.g. Some weights values can go even to zero
- It **implements** a control of the model complexity
- This leads to a model with **less (or proper) *VC-Dim***, with a trade-off obtained through **just a (1) parameter** that you can control: the λ
- **Exercise**: connect Tikhonov regularization with the slide on SLT (see the introduction) to see
 - why this can help to have a better bound on R , and
 - how lambda values can rule the underfitting/overfitting cases
- **Exercise**: derive the new learning rule with weight decay using the Tikhonov loss: computing again the partial derivatives of $Loss$ with respect to w , separating data term ($\bullet \eta$) and penalty term ($\bullet \lambda$)

Slide from Intro slides: 9th Order Polynomial



Dip. Informatica
University of Pisa

As to have Lambda 0



We have already seen it
 $E(\mathbf{w}) = 0$ on training data and **overfitting**

Regularization effect

$$\ln \lambda = -18$$

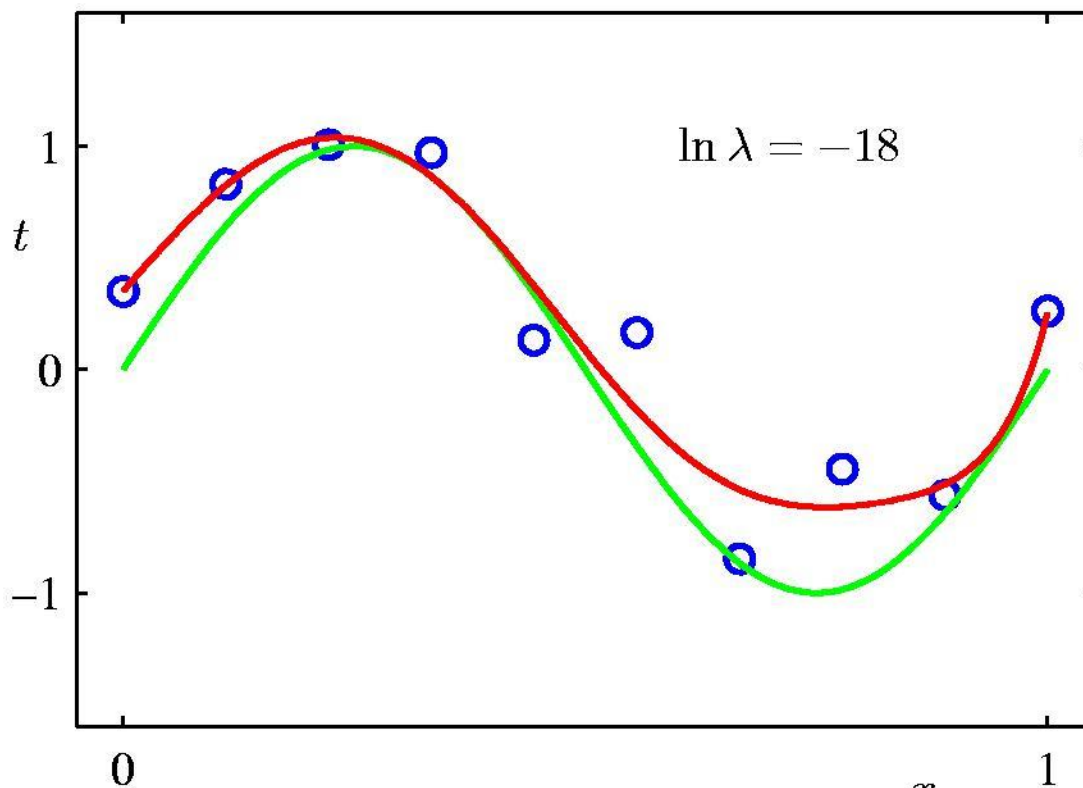


Dip. Informatica
University of Pisa

- 9th Order Polynomial

Lambda small positive

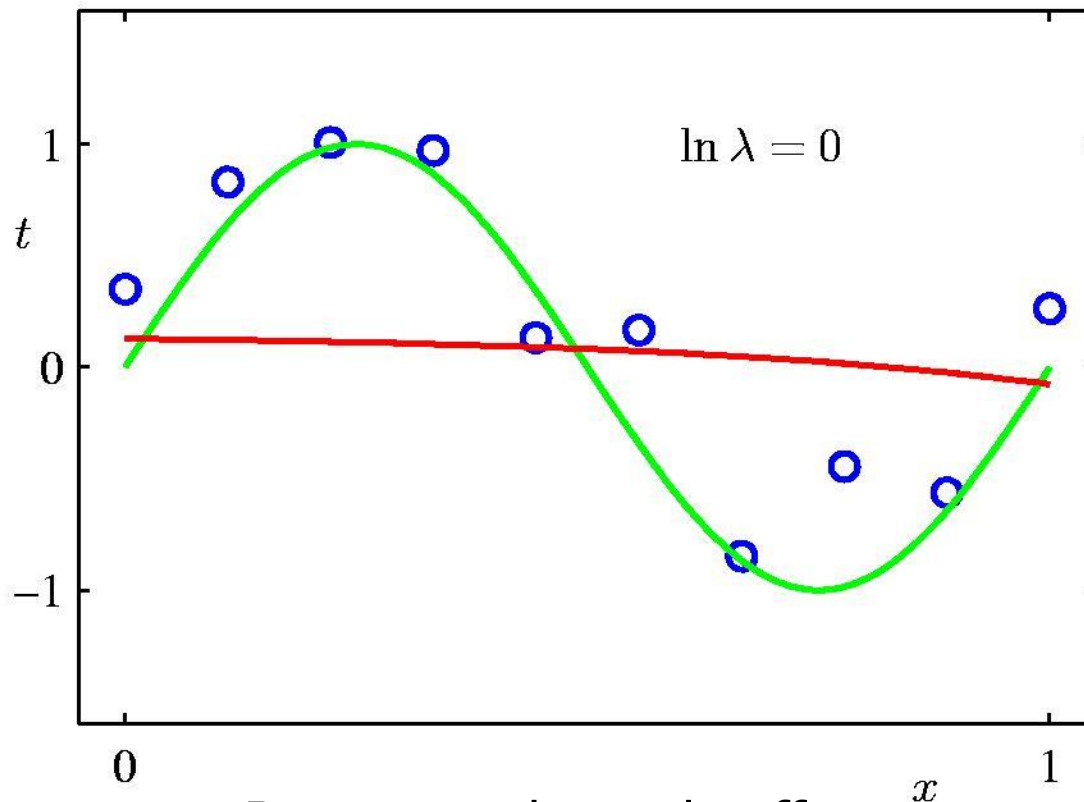
$$\lambda = 0.0000000152$$



Once regularized with a suitable value of lambda it works well!
It is worse on training data but we reduce the **overfitting**

Too High lambda ($\ln \lambda = 0$)

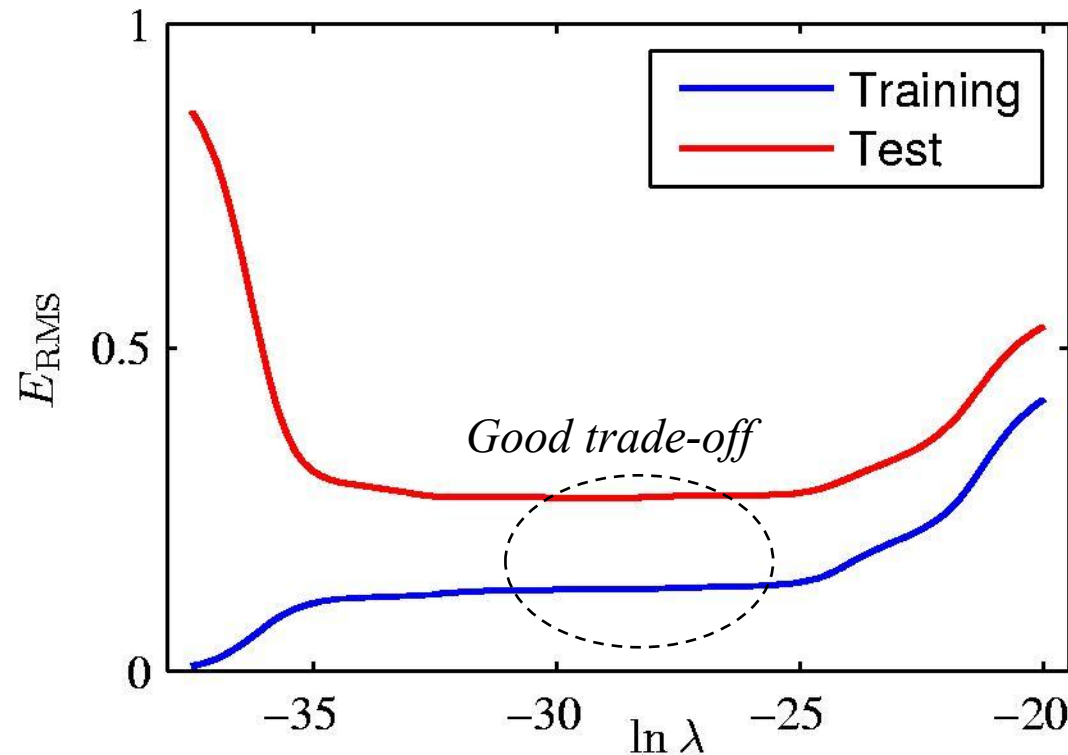
Very high lambda, close to 1



But we need a trade-off ...

(because a too high lambda leads as to **underfitting**)

Regularization: E_{RMS} vs $\ln \lambda$



Low lambda $\rightarrow$ overfitting

High lambda $\rightarrow$ underfitting

Polynomial Coefficients

	$\ln \lambda = -\infty$	$\ln \lambda = -18$	$\ln \lambda = 0$
w_0^*	0.35	0.35	0.13
w_1^*	232.37	4.74	-0.05
w_2^*	-5321.83	-0.77	-0.06
w_3^*	48568.31	-31.97	-0.05
w_4^*	-231639.30	-3.89	-0.03
w_5^*	640042.26	55.28	-0.02
w_6^*	-1061800.52	41.32	-0.01
w_7^*	1042400.18	-45.95	-0.00
w_8^*	-557682.99	-91.53	0.00
w_9^*	125201.43	72.68	0.01

Other Regularization Tech. for Linear Models



Dip. Informatica
University of Pisa

- Ridge regression ($\| \cdot \|_2$)
 - Lasso ($\| \cdot \|_1$)
 - Elastic nets (use both $\| \cdot \|_1$ and $\| \cdot \|_2$)
-
- We will introduce them later in the course (or in CM course), anyway:
 - The *L2 norm* penalizes the square value of the weight and tends to drive all the weights to smaller values.
 - On the other hand, the *L1 norm* penalizes the absolute value of the weights and tends to drive some weights to exactly zero (while allowing some weights to be large) → toward feature selection!
 - Unfortunately $\| \cdot \|_1$ (absolute value) introduce a non differentiable loss → needs other approaches



Other Improvements (optional)



Dip. Informatica
University of Pisa

- Data augmentation:
 - *Inputs with added noise*: teaching models to ignore irrelevant variations, it contributes to robustness and *regularization*
 - Oversampling to address imbalanced datasets
 - ...
- Derived inputs

a small number of new variables is used in place of the $\mathbf{x}$ inputs, which are a linear combination of $\mathbf{x}$.

 - Principal Component Regression
 - Partial Least Squares



Multi-class task (#) (optional)



Dip. Informatica
University of Pisa

Two very simple approaches for multi-class:

- Class 1-of-K rep: {red, green, blue} $\rightarrow$ (0,0,1), (0,1,0), (1,0,0). $\rightarrow$ solve 3 linear models
- **OVA**: "one-vs-all": a discriminant function for each class

built on top of real-valued binary classifiers

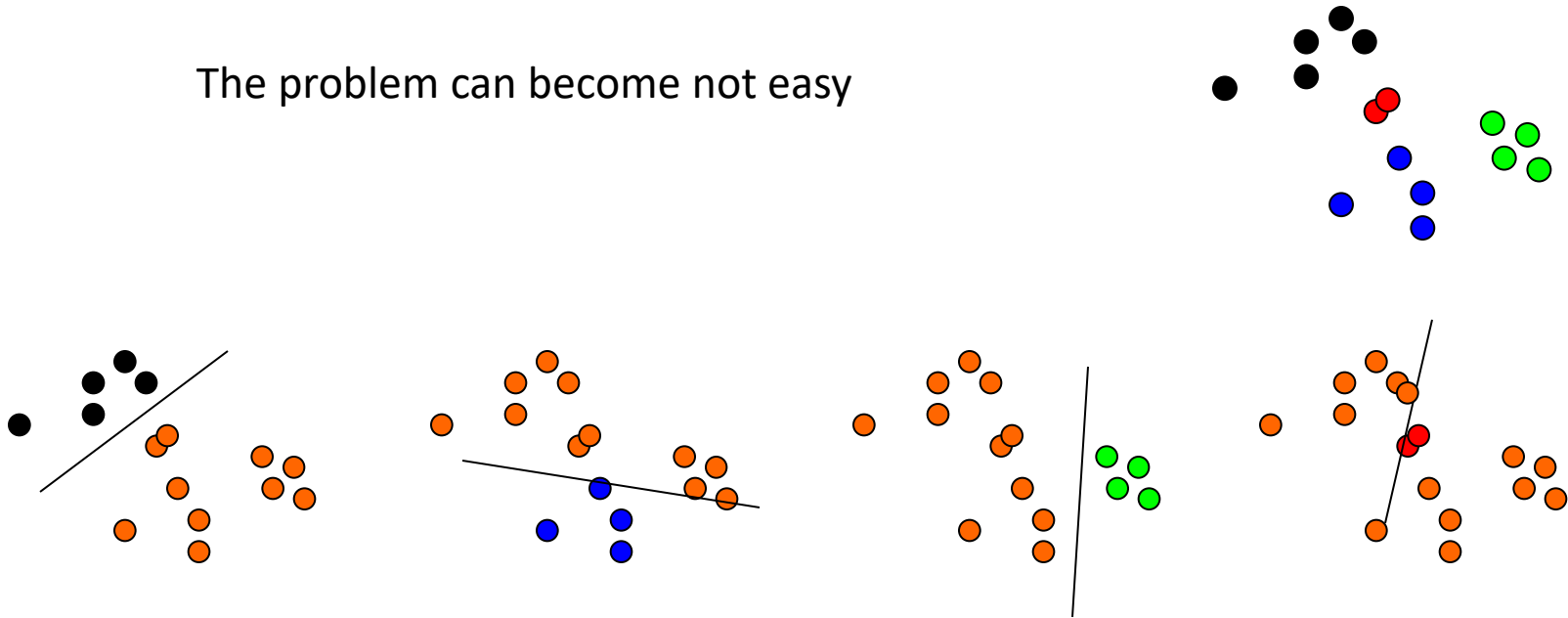
- train K different binary classifiers, each one trained to distinguish the examples in a single class from the examples in all remaining classes.
- to classify a new example, the K classifiers are run, and the classifier which outputs the largest (most positive) value is chosen.
- **AVA**: "all-vs-all" = "one-versus-one":
 - each classifier separates a pair of classes. Apply it to all the pairs: $K(K-1)$
[How many training?*
 - to classify a new example, all the classifiers are run, and the winner is the one with the max sum of outputs versus all the other classes **OR** the class with most votes
 - training data set for each classifier is much smaller

Criticism (##) (optional)



Dip. Informatica
University of Pisa

The problem can become not easy



- Masking: classes can be masked by others (for high K)
- A wide array of more sophisticated approaches for multiclass classification exists ...
- Some models can deal directly with multi-output

Other learner models for classification (#) (optional)



- Linear Discriminant Analysis (also multi-class)
- Logistic regression
 - $P(y|\mathbf{x})$ starting from modeling the class density as a know density

Extensions: ML Models

(pro future) (#)



Dip. Informatica
University of Pisa

In ML course (read it later!):

- Perceptron (Rosenblatt 1958, biological inspiration):
 - “minimize (only) misclassifications” algorithm
 - basis for Neural Networks (set of units with layers)
 - Adaptive *basis expansions* (*phi learned by training*)
 - Feature representation learning in each layer (deep learning concept)
 - Gradient descent approach for learning
- SVM (Vapnik 1996):
 - *regularization* via the concept of maximum margin: Maximize the gap (margin) between the two classes on the training data.
 - enlarge the feature space via *basis expansions* (e.g. polynomials).
- NN and SVM models realize (also) a flexible ***non-linear*** approximation for classification and regression problems

Lessons Learned

- We concretely showed that is possible to *learn* by tuning free parameters $\mathbf{w}$, searching over a continuous space guided by a loss f .
 - How to state/formulate a regression/classification learning problem for a linear model by LMS
 - How to derive the learning algorithm (basic univariate, direct and **gradient** based forms)
 - Two learning algorithms
 - How the model works after training
 - Linear models have limitations (strong language bias)
- How to extend to non-linear modeling (by **LBE**)
- How to implement a **regularization approach** changing the loss f .
- The role of regularization for the **control of the model complexity**

Also useful for your self assessment (try to check if you can answer)

ML Course structure

Discussion on Linear Model

In the *fil rouge* of ML, 2 main concepts up to now:

- Basis expansion → (implement) more flexibility
- Regularization → (implement) control of complexity
- The cases of the *linear models* provided an *instance* rooted in the classical mathematical approaches but the we will find again these concepts, by different models and implemented in different/similar forms, in the modern ML (e.g. for NN & SVM in our course)!
- Now we move on the other extreme, toward a very flexible approach (K-nn)

Bibliography: Linear and K-nn (I)



Dip. Informatica
University of Pisa

- Hastie, Tibshirani, Friedman, The Elements of Statistical Learning, Springer Verlag, 2001-2017 (*check the new Ed.s*): **chap 2**
(note: it exists an on-line pdf version)
- Mitchell:
 - Linear model and LMS alg.: chap 4.4
 - K-nn: **chap 8**
- Haykin (2nd edition):
 - Linear model and LMS alg.: chap 3 up to 3.7
[details on Newton and Gauss-Newton methods are not strictly needed]
- Haykin (3rd edition):
 - Linear model and LMS alg.: chap 3
[details on Newton and Gauss-Newton and other advanced approaches are not strictly needed]

Bibliography: Linear and K-nn (II)



Dip. Informatica
University of Pisa

- Moreover...
- Further readings (not mandatory!):
 - Gently introduction to linear models:
AIMA (Russel, Norvig, Artificial Intelligence: A Modern Approach), ed .3: **chap 18.6** (18.6.1,18.6.2,18.6.3)

- To go in deep for **Linear least squares**

You can start from www resource as (With nice examples):

https://en.wikipedia.org/wiki/Linear_least_squares_%28mathematics%29

- **LS in general**

http://en.wikipedia.org/wiki/Least_squares

For information

Alessio Micheli
micheli@di.unipi.it

www.di.unipi.it/groups/ciml



Dipartimento di Informatica
Università di Pisa - Italy
Micheli



**Computational Intelligence &
Machine Learning Group**